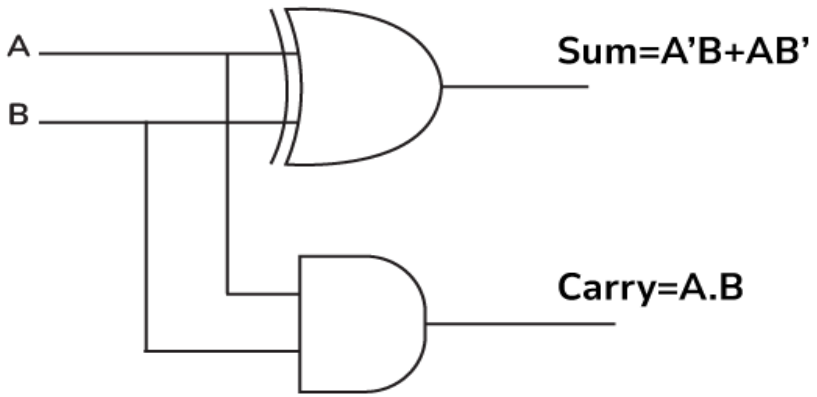
HDL Lab Experiments

Faculty: Dr. Jisha P

Section: B

Half adder

Half Adder



Truth Table

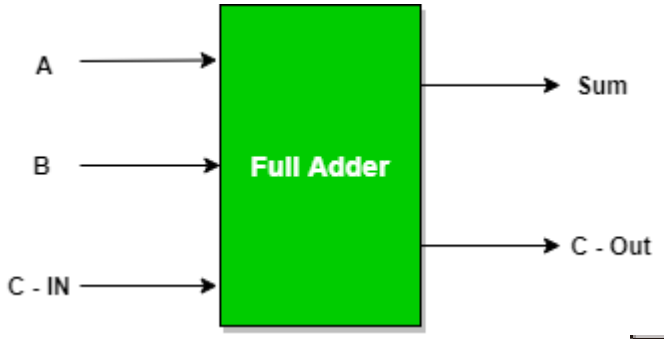
A	B	S	C
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

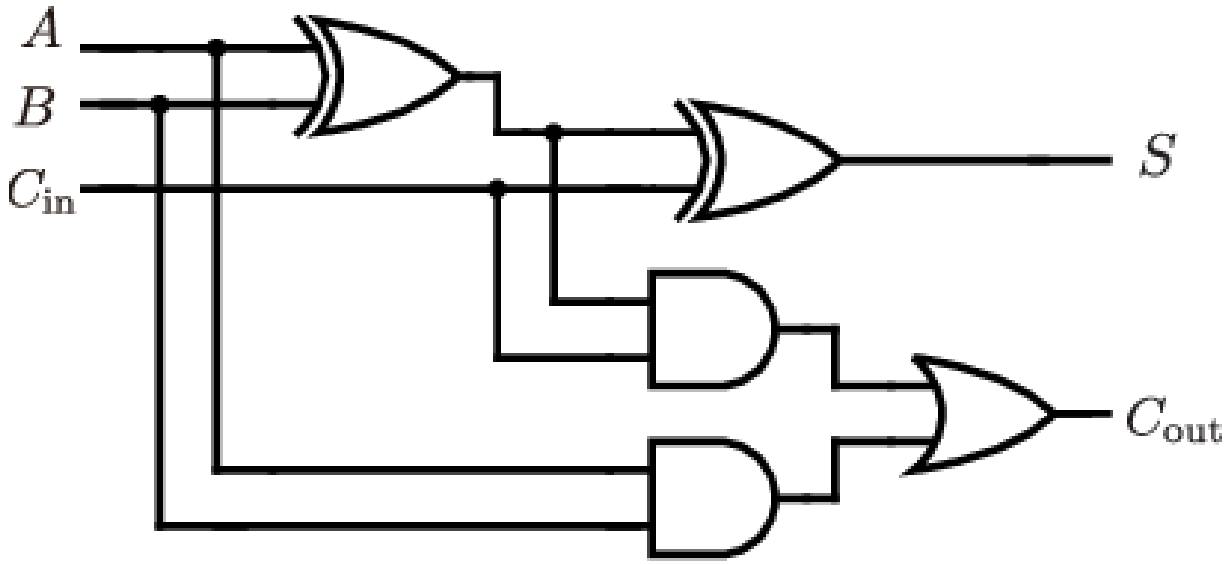
1.a- Gate-level modelling: Half adder

```
module half_adder (

input a,b,

output sum,carry);

xor (sum,a,b);

and (carry,a,b);

endmodule
```

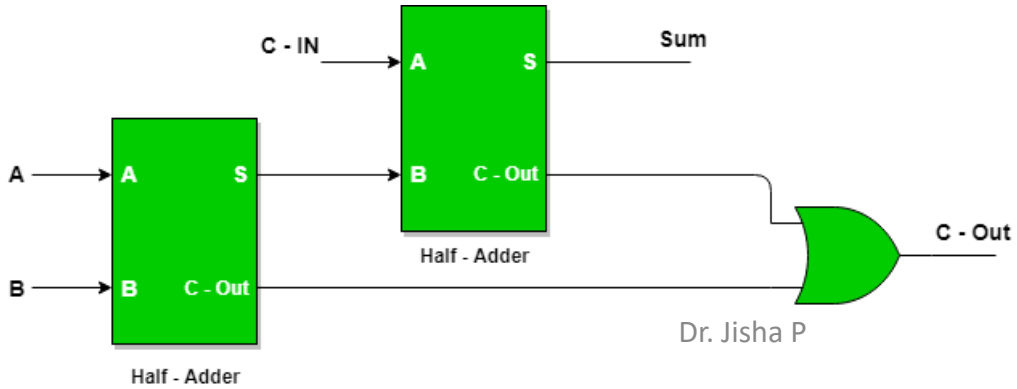
```
module half_adder_tb;
reg a,b;
wire sum,carry;
half_adder ha (a,b,sum,carry);
initial
begin
a=0;b=0;
#10 a=0;b=1;
#10 a=1;b=0;
#10 a=1;b=1;
#10
$finish;
end
endmodule
```

2. Full Adder





Inputs			Outputs	
A	B	C_{in}	S	C_{out}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1



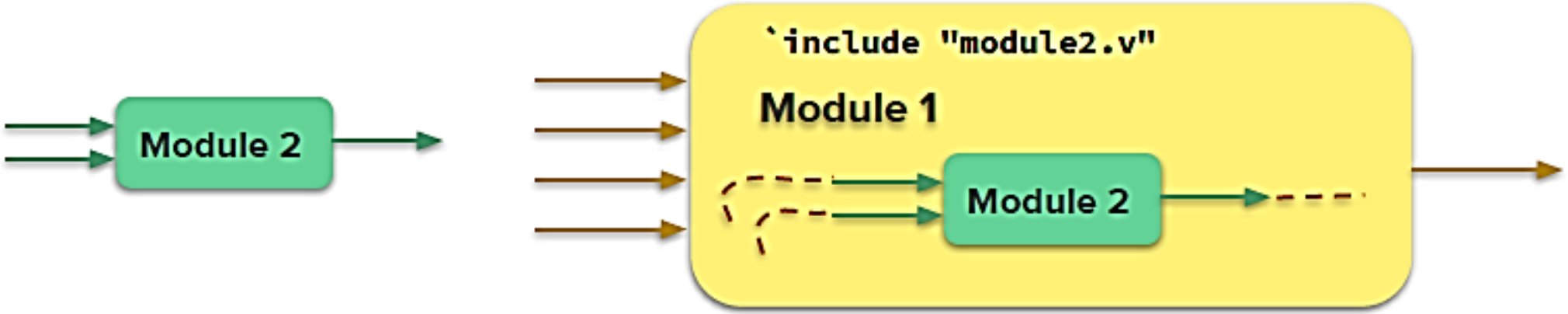
b. Gate-level modelling: Full adder

```
module full_adder (input a,b,c,
output sum,carry);
wire y1,y2,y3;
xor g1 (sum,a,b,c);
and g2 (y1,a,b);
and g3 (y2,b,c);
and g4 (y3,a,c);
or g5 (carry,y1,y2,y3);
endmodule
```

```
module full_adder_tb;
reg a,b,c;
wire sum,carry;
full_adder fa(a,b,c,sum,carry);
initial begin
a=0;b=0;c=0;
#10 a=0;b=0;c=1;
#10 a=0;b=1;c=0;
#10 a=0;b=0;c=1;
#10 a=1;b=0;c=0;
#10 a=1;b=0;c=1;
#10 a=1;b=1;c=0;
#10 a=1;b=1;c=1;
#10 $finish;
end
endmodule
```

Verilog Language Features

Two Ways to Specify Connectivity During Module Instantiation



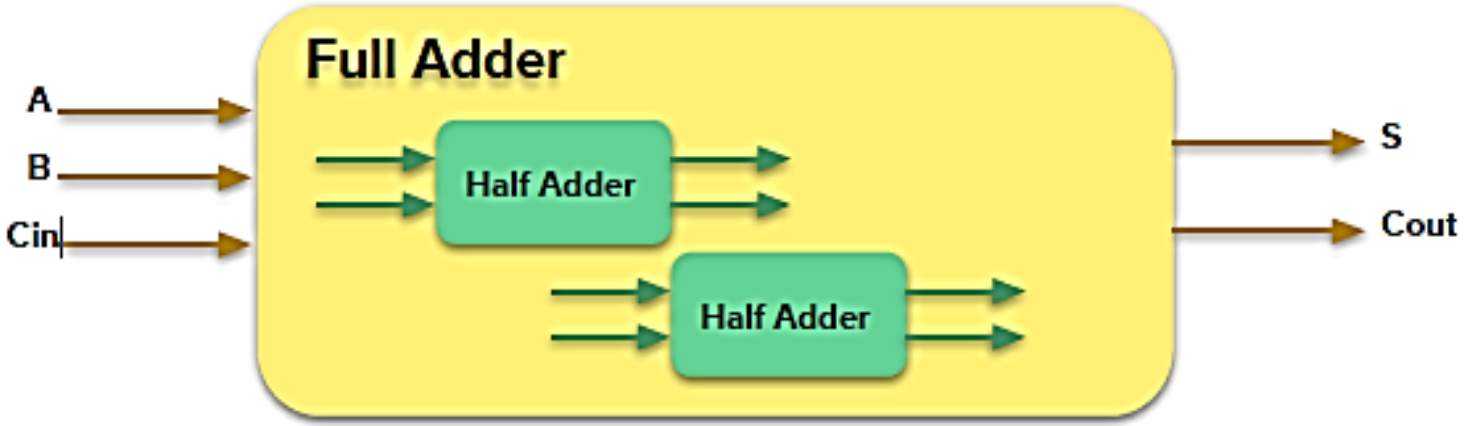
Connectivity of the signal lines between two modules can be specified in 2 ways:

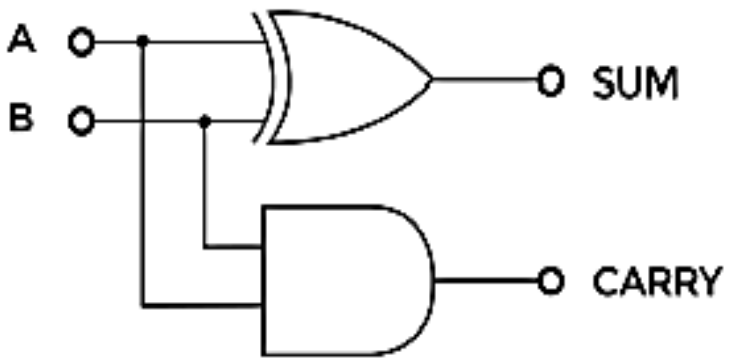
- **Positional Association (Implicit)**
  - Parameters listed **in the same order** as in the original module description.
- **Explicit Association**
  - Parameters **explicitly** listed **in arbitrary order**.

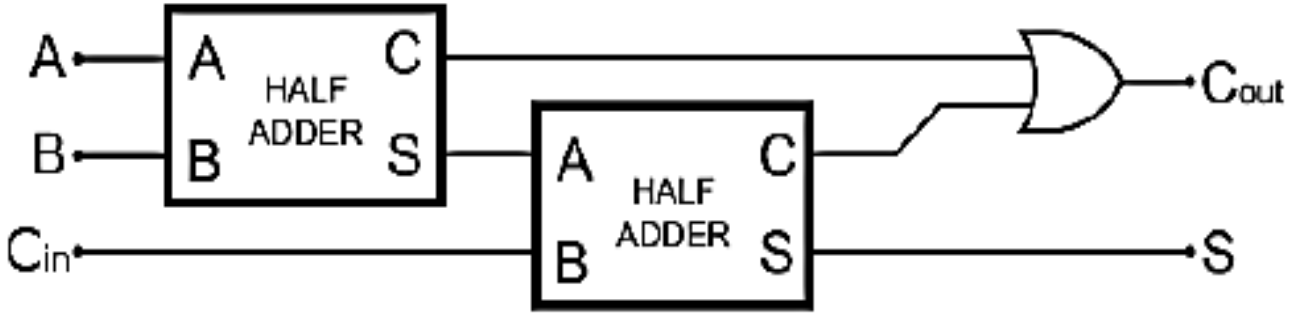
Verilog Language Features

- Positional Association Example - Full Adder Using Half Adder Module

A	B	Sum	Carry
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

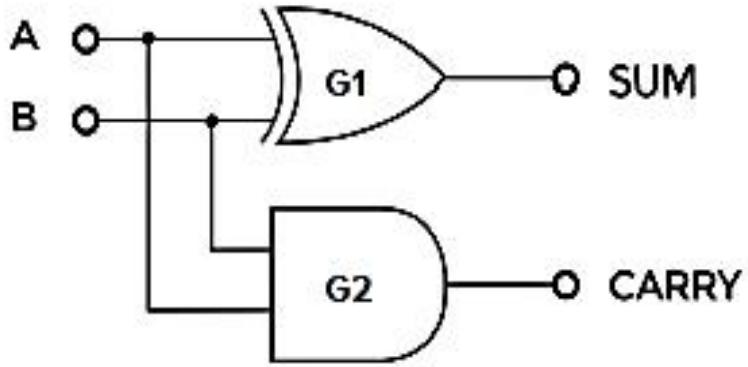






Verilog Language Features

- Positional Association Example - Full Adder Using Half Adder Module




```
module half_adder (Sum, Carry, A, B);
    input A, B;
    output Carry, Sum;
    //structural description
    xor G1(Sum, A, B);
    and G2(Carry, A, B);
endmodule
```

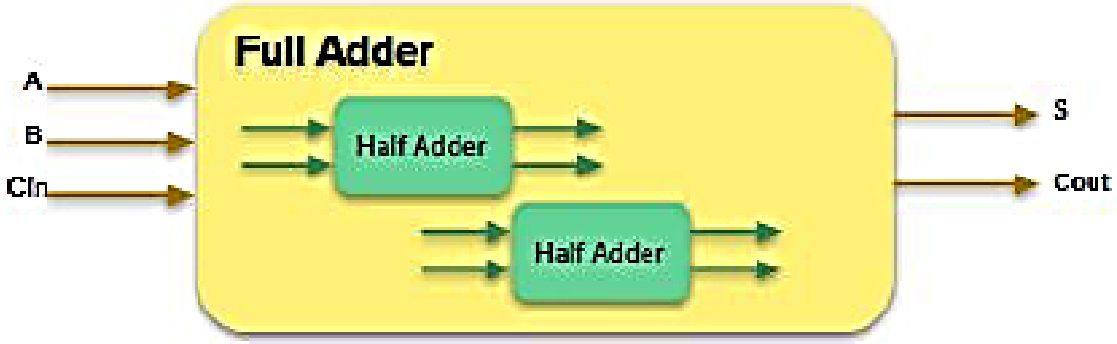
=

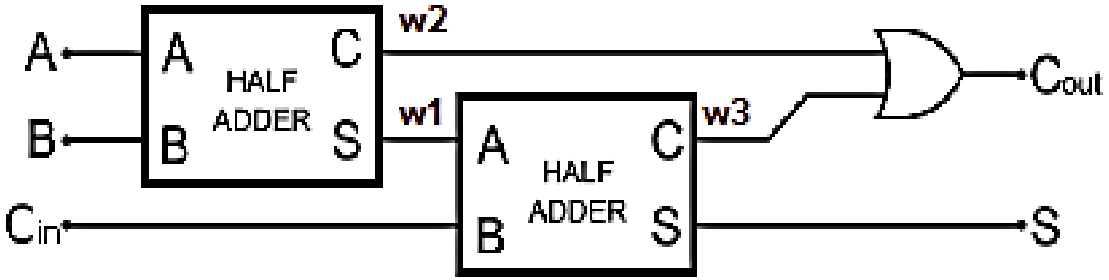
```
//behavioral description
assign Sum = A ^ B;
assign Carry = A & B;
```

Verilog Language Features

Note the port order

- Positional Association Example - Full Adder Using Half Adder Module



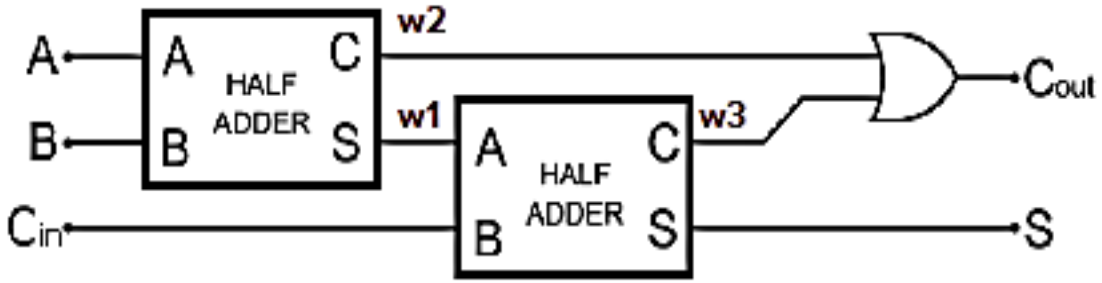



```
module half_adder (Sum, Carry, A, B);
```

```
module full_adder (Sum, Cout, A, B, Cin);
    input A, B, Cin;
    output Cout, Sum;
    wire w1, w2, w3;
    half_adder HA1 (w1, w2, A, B);
    half_adder HA2 (Sum, w3, Cin, w1);
    or (Cout, w2, w3);
endmodule
```

Verilog Language Features

- Explicit Association Example - Full Adder Using Half Adder Module



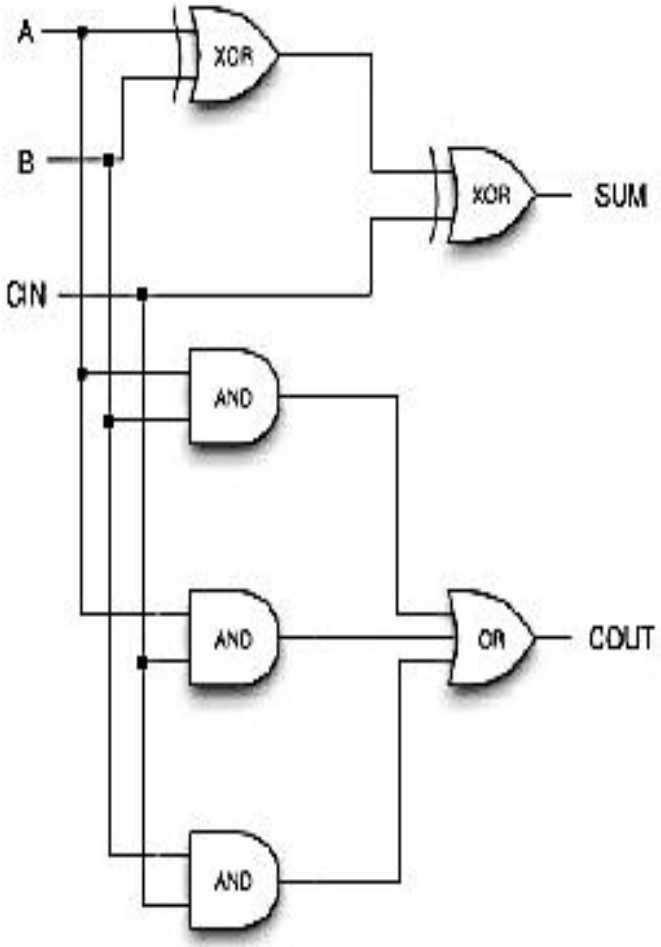

```
module half_adder (Sum, Carry, A, B);
```

Ports are explicitly specified - order is not important

```
module full_adder (Sum, Cout, A, B, Cin);
    input A, B, Cin;
    output Cout, Sum;
    wire w1, w2, w3;
    half_adder HA1 (.A(A), .B(B), .Sum(w1), .Carry(w2));
    half_adder HA2 (.Sum(Sum), .Carry(w3), .B(Cin), .A(w1));
    or (Cout, w2, w3);
endmodule
```

Less chance for errors

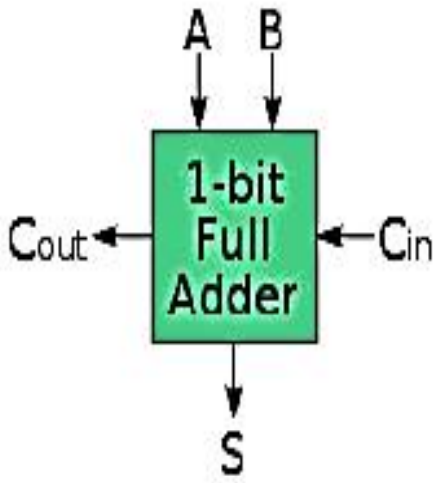
A	B	Cin	S	Cout
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1




```
sum = (A^B)^C
Cout = AB+BC+AC
```

Full_Adder.v module

```
module Full_Adder(
    input A,
    input B,
    input Cin,
    output S,
    output Cout
    );
    assign S = (A^B)^Cin;
    assign Cout = (A&B)|(B&Cin)|(Cin&A);
endmodule
```



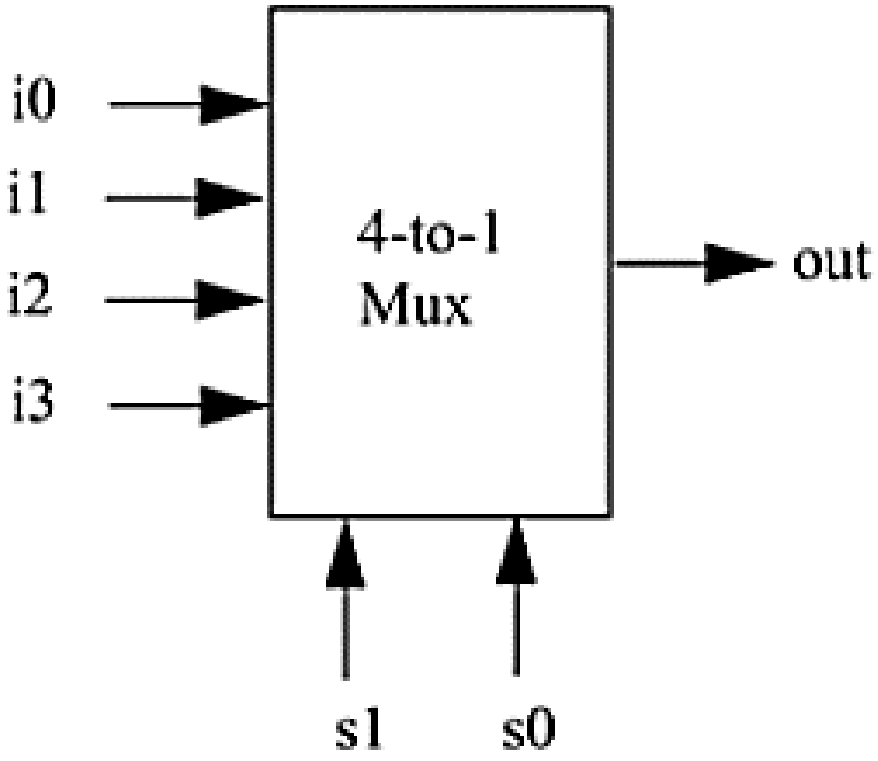
Note the behavioral description

Simulation results:

2. Gate-level modelling: **Multiplexers** and demultiplexers

https://prezi.com/view/6jxQNVPgPdGqST6i9Xzk/

4-to-1 Multiplexer I/O Diagram



Truth Table

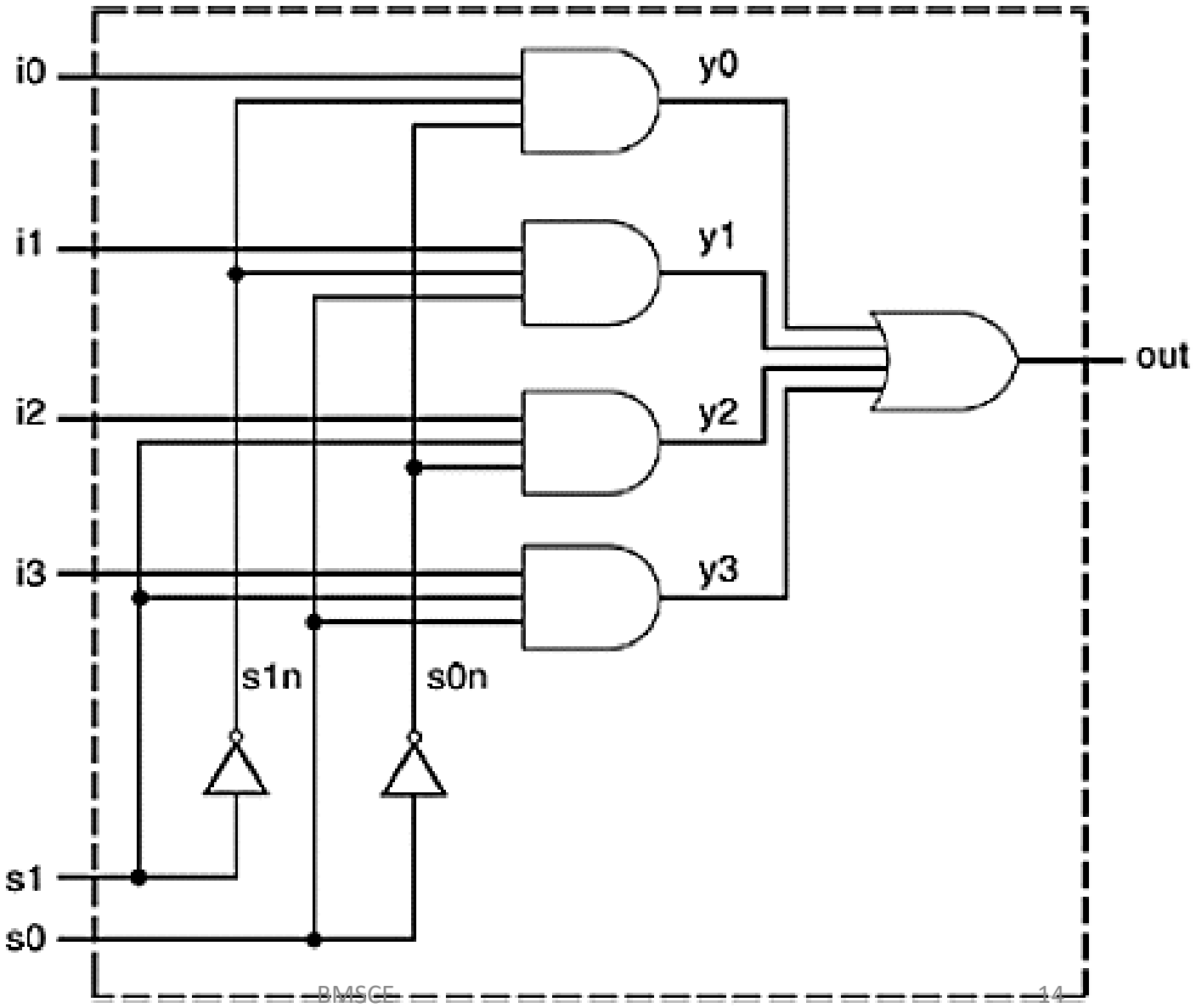
S0	S1	Y
0	0	I0
0	1	I1
1	0	I2
1	1	I3

So, final equation,

$Y = S0'.S1'.I0 + S0'.S1.I1 + S0.S1'.I2 + S0.S1.I3$

4-to-1 Multiplexer Logic Diagram

- The logic diagram has a one-to-one correspondence with the Verilog description.
- Two intermediate nets, **s0n and s1n**, are created; they are complements of input signals **s1 and s0**.
- Internal nets **y0, y1, y2, y3** are also required.
- Instance names are optional for Verilog primitives but are mandatory for instances of user-defined modules.



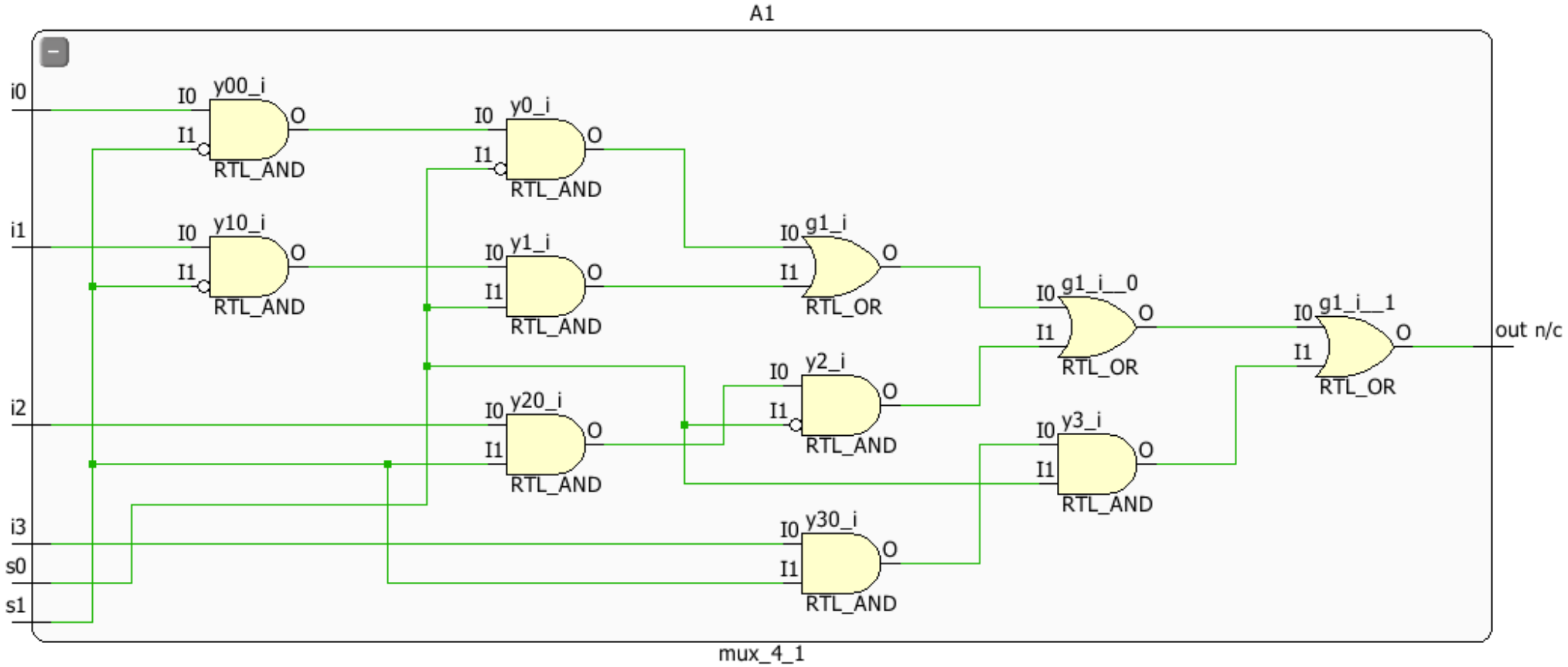
Structural modelling (gate level): 4:1 Multiplexer

```
module mux_4_to_1(output out, input
i0,i1,i2,i3,s1,s0 );
wire s1n, s0n; //internal wire declarations
wire y0, y1, y2, y3;
not(s1n,s1);
not(s0n,s0);
and(y0,i0,s1n,s0n); //3-input and gates instantiated
and(y1,i1,s1n,s0);
and(y2,i2,s1,s0n);
and(y3,i3,s1,s0);
or(out,y0,y1,y2,y3); //4-input or gates instantiated
endmodule
```

```
module tb;
wire out;
reg i0,i1,i2,i3,s1,s0;
mux_4_to_1 A1(out,i0,i1,i2,i3,s1,s0);
initial
begin
s1=0;s0=0;i0=0;i1=1;i2=1;i3=1;
#10 i0=1;i1=0;i2=0;i3=0;
#15 s0=1;i0=1;i1=0;i2=1;i3=1;
#5 i0=0;i1=1;i2=0;i3=0;
#10 s0=0;s1=1;i0=1;i1=1;i2=0;i3=1;
#5 i0=0;i1=0;i2=1;i3=0;
#10 s0=1;i0=1;i1=1;i2=1;i3=0;
#5 i0=0;i1=0;i2=0;i3=1;
#40 $finish;
end
endmodule
```

Simulation results:

Schematic


A1
i0
i1
i2
i3
s0
s1
y00_i
y10_i
y0_i
y1_i
y20_i
y30_i
g1_i
g1_i__0
g1_i__1
y2_i
y3_i
I0
I1
O
RTL_AND
RTL_OR
out n/c
mux_4_1

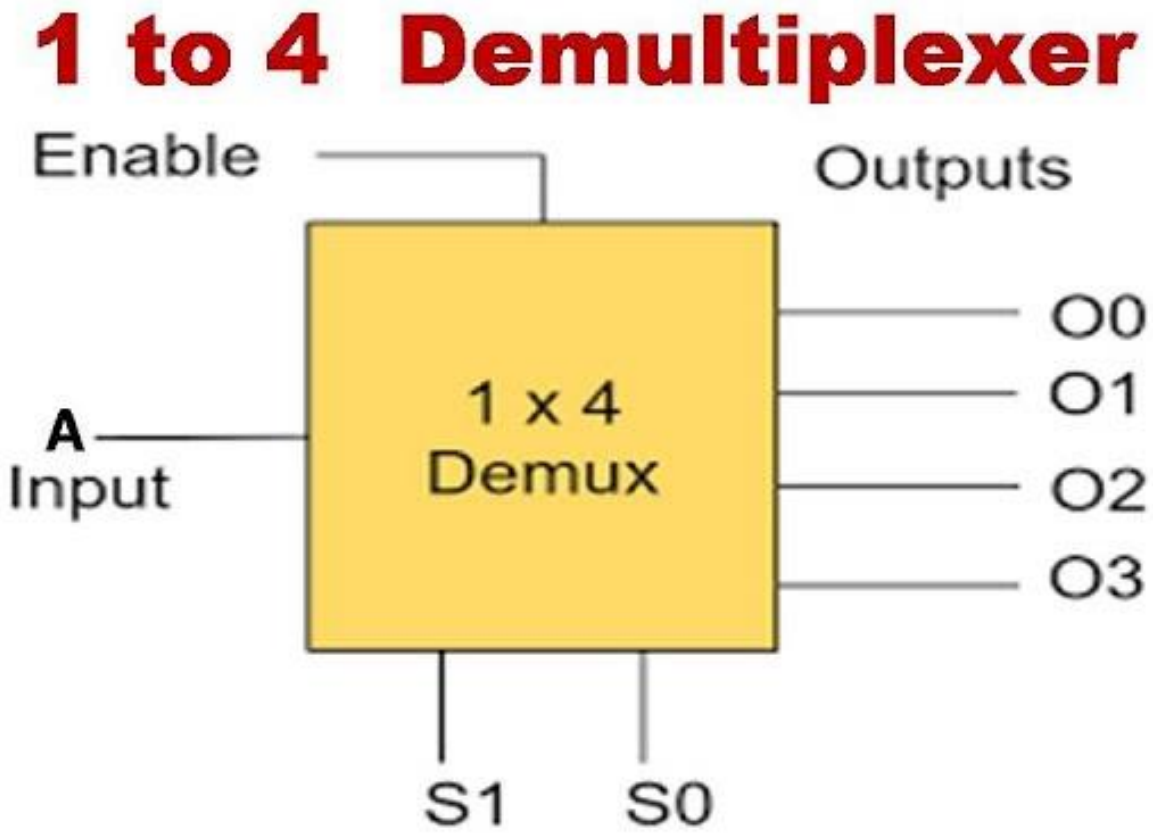
Data flow description modelling: **4-to-1 Multiplexer**

```
module mux_4_to_1( output out, input i0,i1,i2,i3,s1,s0);

Assign y = ((~S[1])&(~s[0])&I[0])|((~S[1])&(S[0])&I[1])|((S[1])&(~S[0])&I[2])|((S[0])&(S[1])&I[3]);

endmodule

//TEST BENCH REMAINS THE SAME
```

Behavioral modelling: **4-to-1 Multiplexer**

```
module mux_4_to_1( output reg out, input i0,i1,i2,i3,s1,s0 );
always @(i0,i1,i2,i3,s1,s0)
case ({s1,s0}) //{} concatenation
2'b00:out=i0;
2'b01:out=i1;
2'b10:out=i2;
2'b11:out=i3;
default:out=1'b0;
endcase
endmodule
//TEST BENCH REMAINS THE SAME
```

2. Gate-level modelling: Multiplexers and **Demultiplexers**

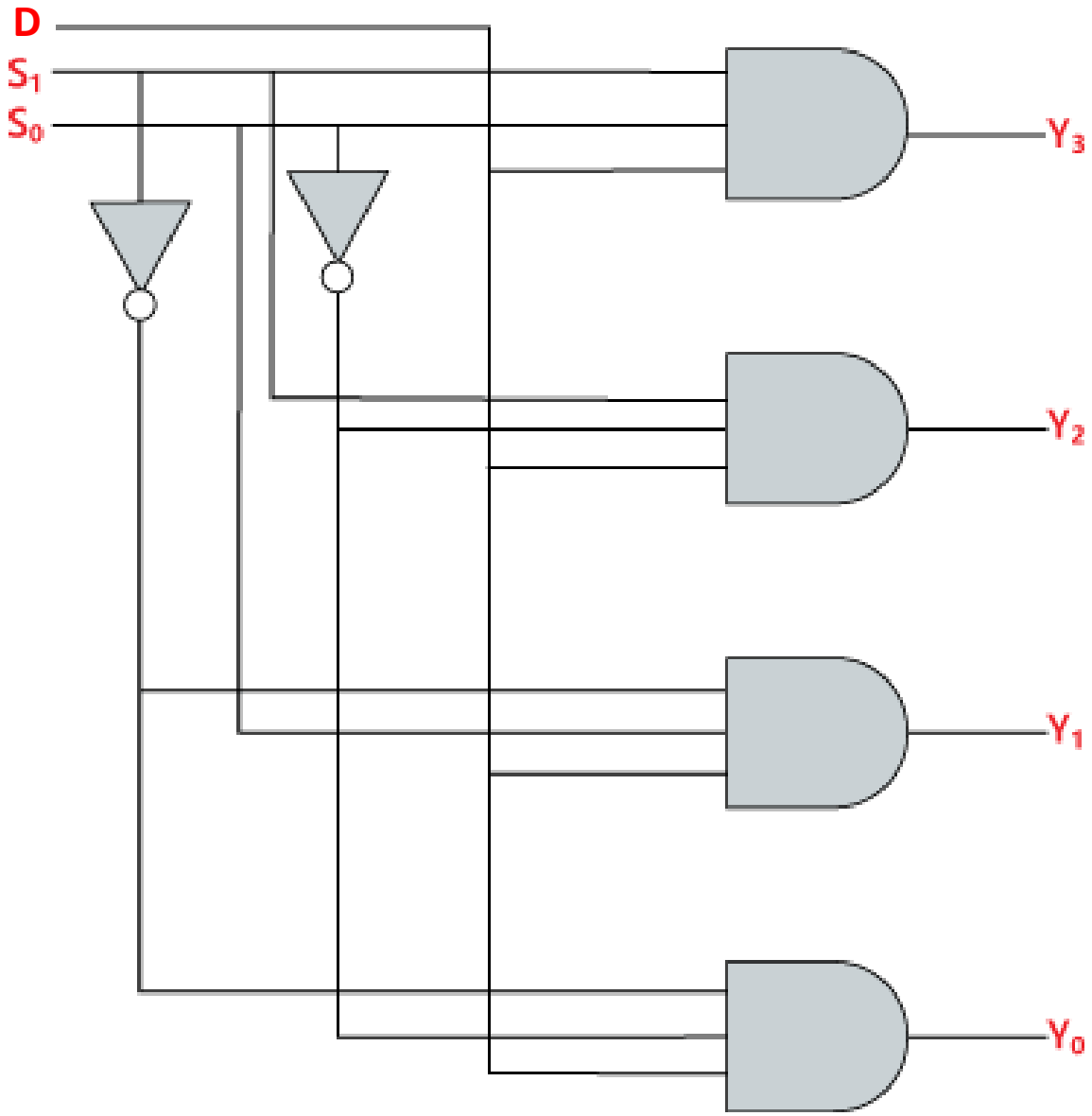


2.b Gate-level modelling: 1:4 **Demultiplexer**

S1	S0	D	Y3	Y2	Y1	Y0
0	0	0	0	0	0	**0**
0	0	1	0	0	0	**1**
0	1	0	0	0	**0**	0
0	1	1	0	0	**1**	0
1	0	0	0	**0**	0	0
1	0	1	0	**1**	0	0
1	1	0	**0**	0	0	0
1	1	1	**1**	0	0	0

$Y0 = \overline{S1}\ \overline{S0}\ D$ $Y1 = \overline{S1}\ S0\ D$ $Y2 = S1\ \overline{S0}\ D$ $Y3 = S1\ S0\ D$

2.b Gate-level modelling: 1:4 Demultiplexer



1:4 Demux: Structural modelling (gate level)

```
module demux(
input i,s1,s0,output o0,o1,o2,o3);
not(s1n,s1);
not(s0n,s0);
and(o0,s1n,s0n,i);
and(o1,s1n,s0,i);
and(o2,s1,s0n,i);
and(o3,s1,s0,i);
or(out,o0,o1,o2,o3);
endmodule
```

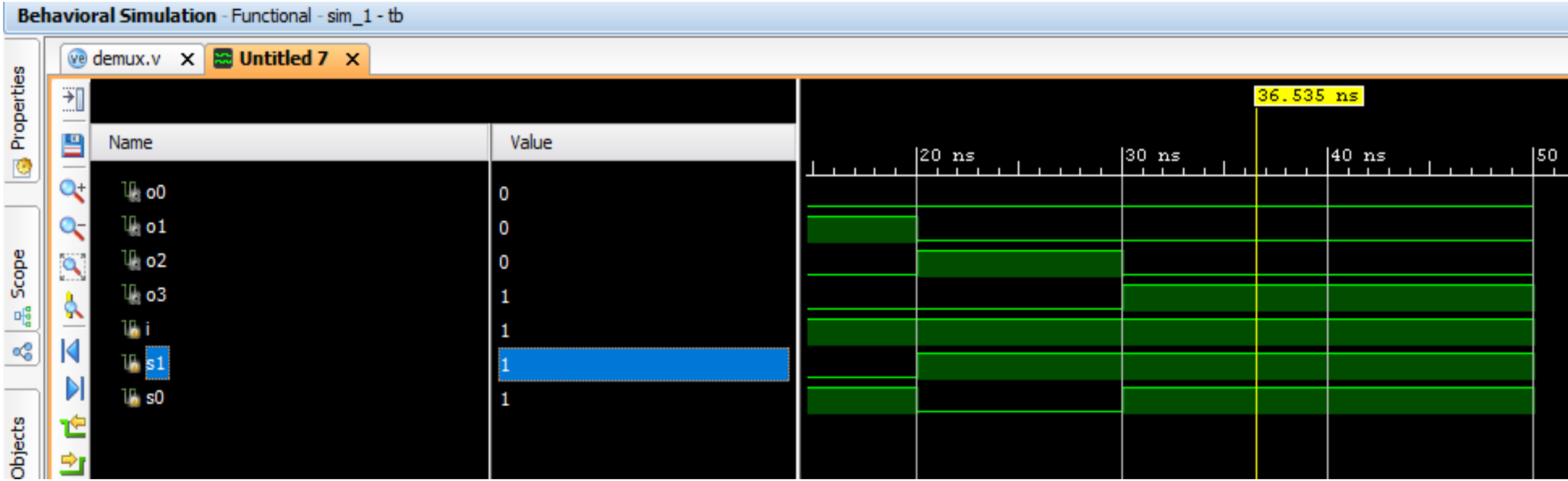
```
module tb;
wire o0,o1,o2,o3;
reg i,s1,s0;

demux A1(i,s1,s0,o0,o1,o2,o3);
initial
begin
s1=0;s0=0;i=1;
#10 s1=0;s0=1;i=1;
#10 s1=1;s0=0;i=1;
#10 s1=1;s0=1;i=1;
#10 $finish;
end
endmodule
```

1:4 Demux: Behavioural modelling

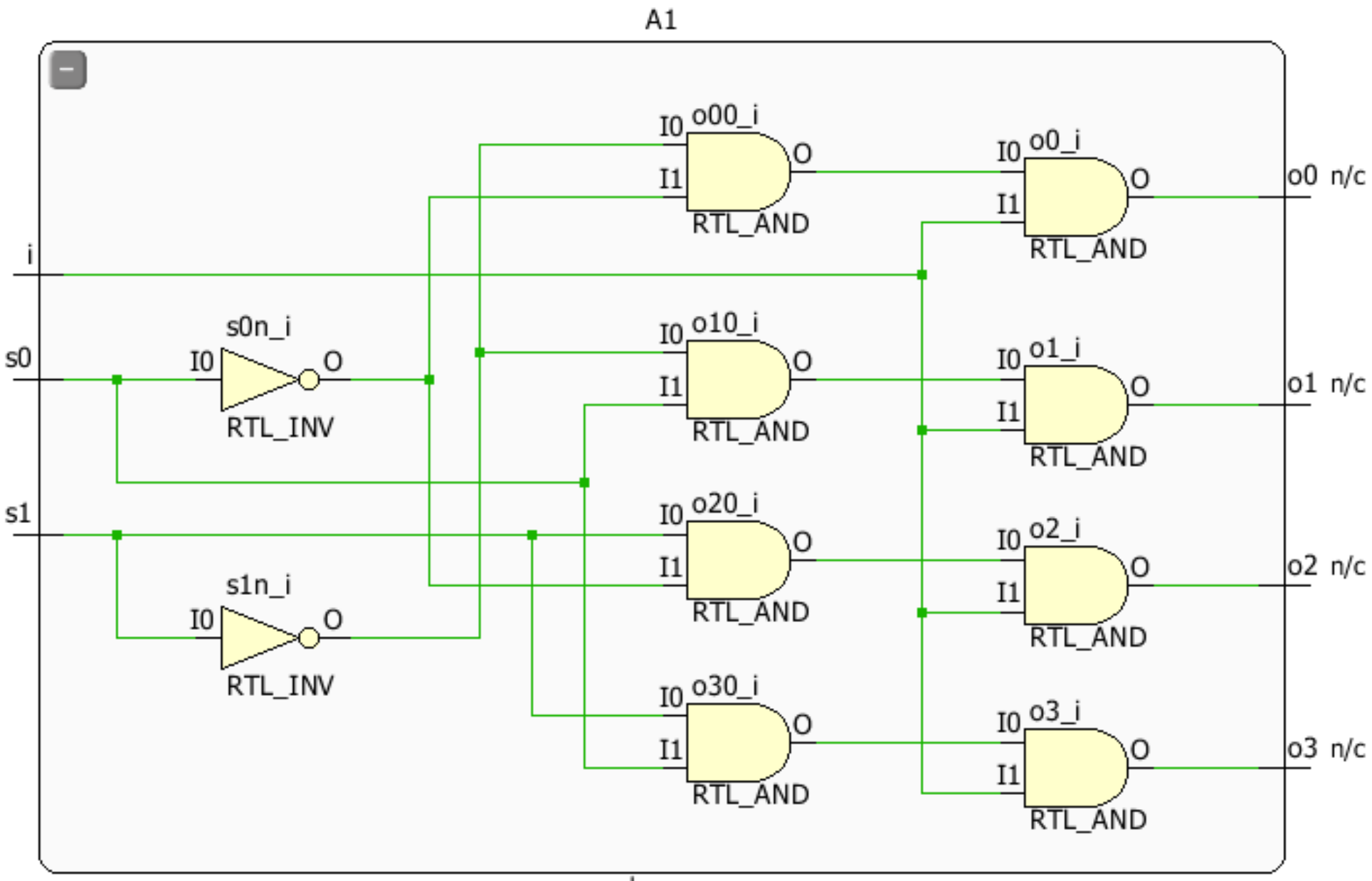
```
module demux( input i,s1,s0,
output o0,o1,o2,o3 );
always @(i,s1,s0)
case ({s1,s0}) //{} concatenation
2'b00:o0=i;
2'b01:o1=i;
2'b10:o2=i;
2'b11:o3=i;
default:o0=1'b0;
endcase
endmodule
```

```
module tb;
wire o0,o1,o2,o3;
reg i,s1,s0;

demux A1(i,s1,s0,o0,o1,o2,o3);
initial
begin
s1=0;s0=0;i=1;
#10 s1=0;s0=1;i=1;
#10 s1=1;s0=0;i=1;
#10 s1=1;s0=1;i=1;
#10 $finish;
end
endmodule
```

Simulation results:



Schematic:



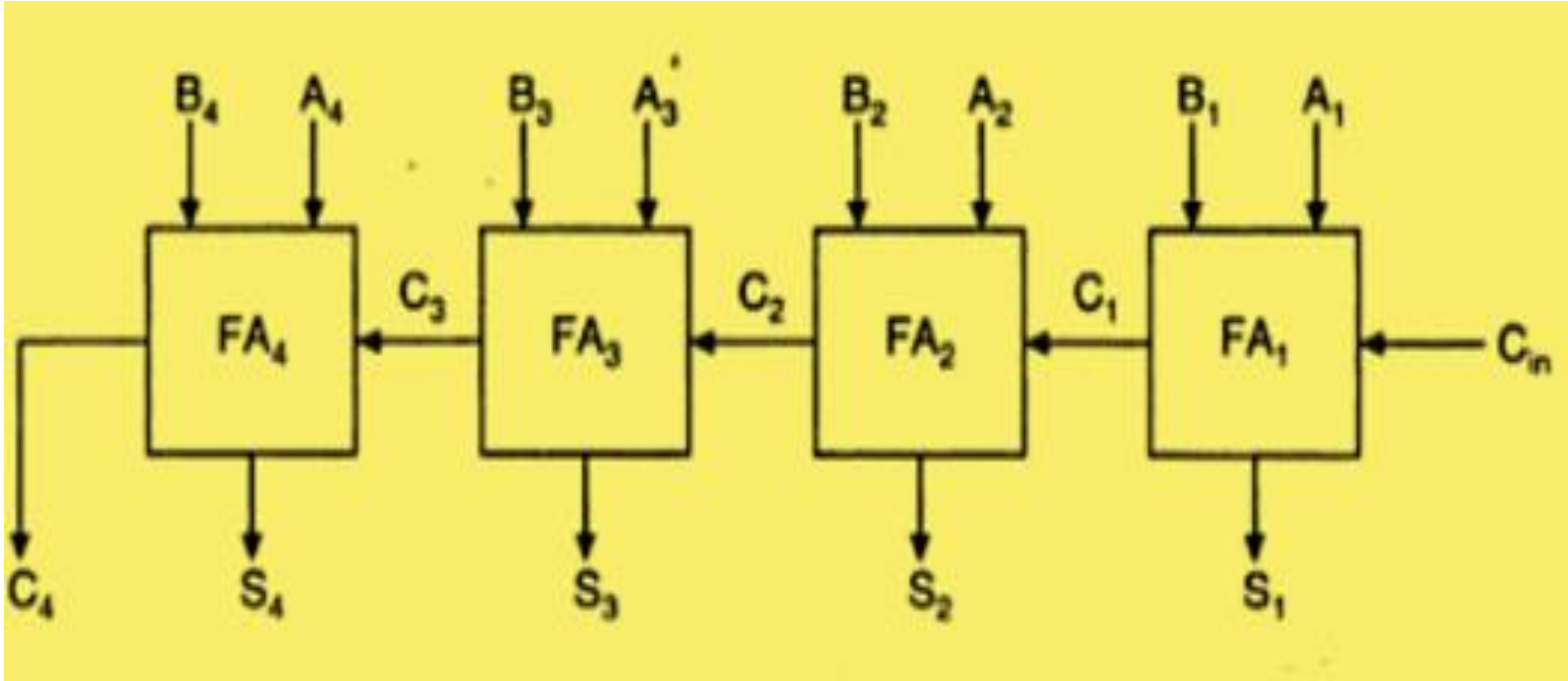
demux

EXP- 3: Ripple Carry adder

- **“Ripple Carry Adder”** - It’s a unique type of logic circuit used for adding the N-bit numbers in digital operations.
- A structure of multiple full adders is cascaded in a manner to gives the results of the addition of an n bit binary sequence.
- This adder includes cascaded full adders in its structure so, the carry will be generated at every full adder stage in a ripple-carry adder circuit.
- These carry output at each full adder stage is forwarded to its next full adder and there applied as a carry input to it.
- This process continues up to its last full adder stage.
- So, each carry output bit is rippled to the next stage of a full adder.
- By this reason, it is named as **“RIPPLE CARRY ADDER”**.

EXP- 3: 4-bit Ripple Carry Adder

- In this adder, four full adders are connected in cascade.
- Co is the carry input bit and it is zero always.
- When this input carry ‘Co’ is applied to the two input sequences A1 A2 A3 A4 and B1 B2 B3 B4 then output represented with S1 S2 S3 S4 and output carry C4.

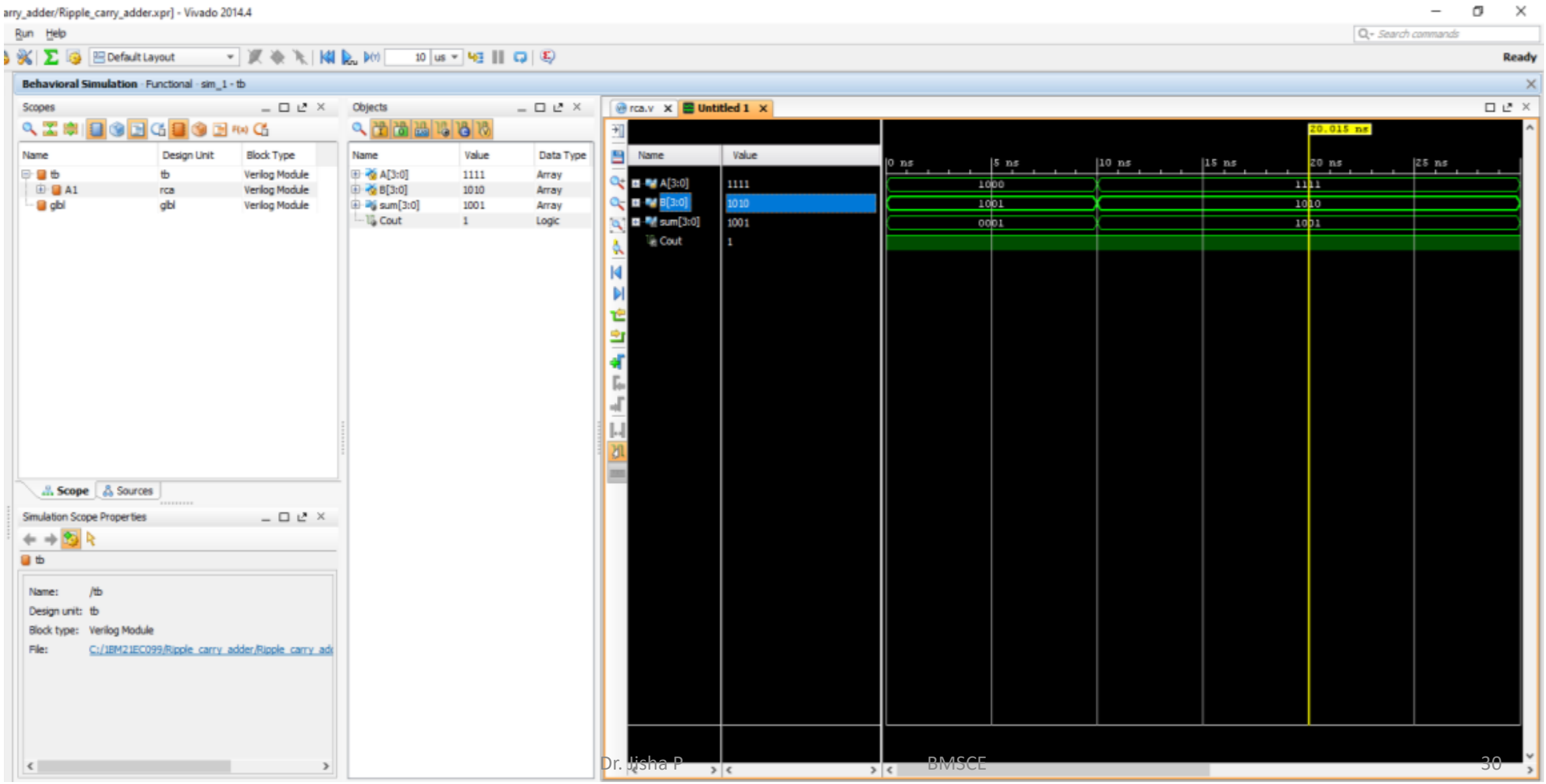


EXP-3: Data flow modelling: Ripple Carry adder

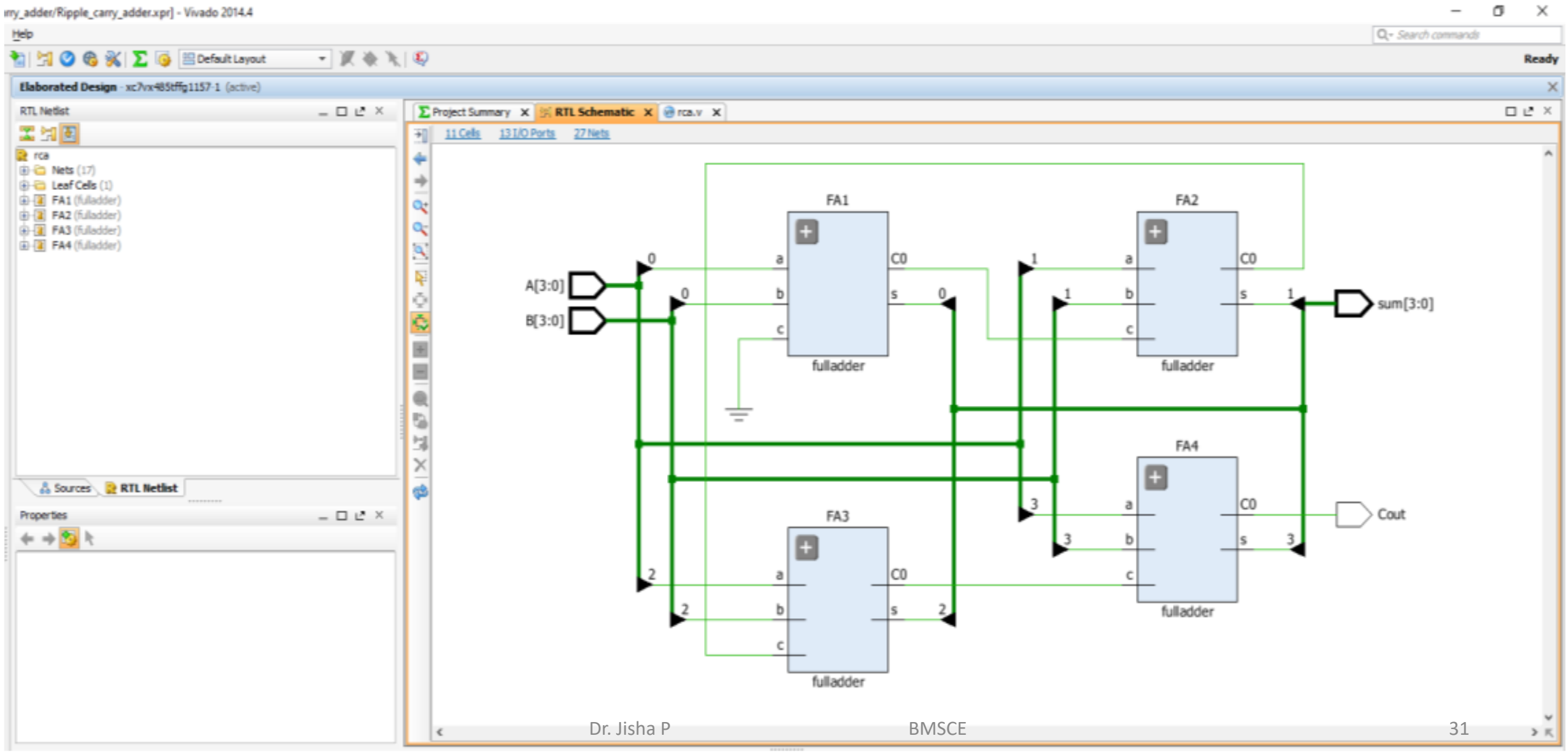
```
module rca( input [3:0]A,B,
output [3:0]sum,
output Cout);
wire [3:0]C;
assign C[0]=0;
fulladder FA1(A[0],B[0],C[0],sum[0],C[1]);
fulladder FA2(A[1],B[1],C[1],sum[1],C[2]);
fulladder FA3(A[2],B[2],C[2],sum[2],C[3]);
fulladder FA4(A[3],B[3],C[3],sum[3],Cout);
endmodule
```

```
module tb;
reg [3:0]A,B;
wire [3:0]sum;
wire Cout;
rca A1(A,B,sum,Cout);
Initial   begin
A=4'd8;B=4'd9;
#10 A=4'hF;B=4'hA;
#20 $finish;
end
endmodule
module fulladder(input a,b,c,output s,C0);
xor(s,a,b,c);
and(y1,a,b);
and(y2,b,c);
and(y3,c,a);
or(C0,y1,y2,y3);
endmodule
```

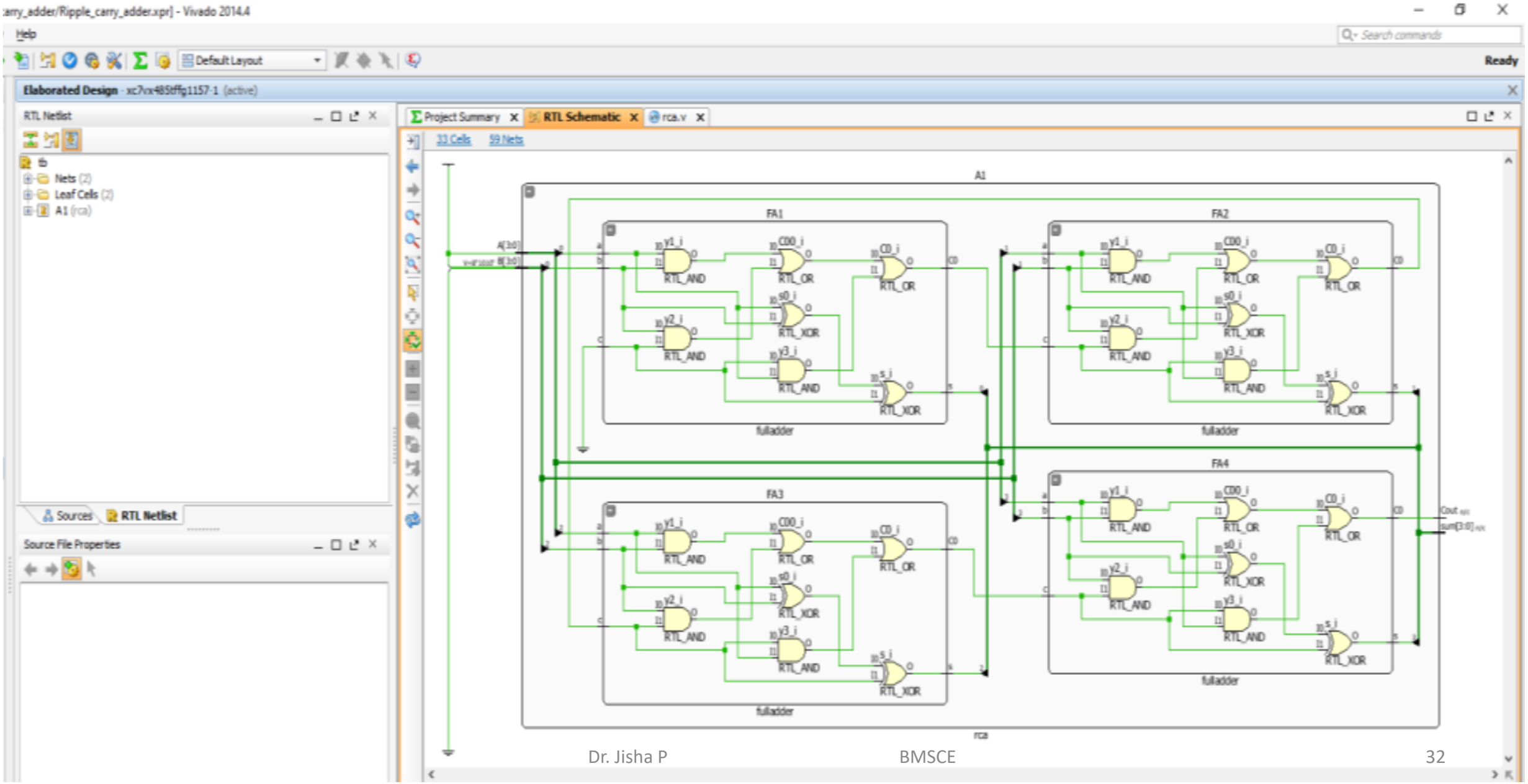
Simulation


arry_adder/Ripple_carry_adder.xpr] - Vivado 2014.4
Run Help
Default Layout
Ready
Behavioral Simulation - Functional - sim_1 - tb
Scopes
Name Design Unit Block Type
tb tb Verilog Module
A1 rca Verilog Module
glbl glbl Verilog Module
Scope Sources
Simulation Scope Properties
tb
Name: /tb
Design unit: tb
Block type: Verilog Module
File: C:/1BM21EC099/Ripple_carry_adder/Ripple_carry_ad
Objects
Name Value Data Type
A[3:0] 1111 Array
B[3:0] 1010 Array
sum[3:0] 1001 Array
Cout 1 Logic
rca.v
Untitled 1
20.015 ns
Name Value
A[3:0] 1111
B[3:0] 1010
sum[3:0] 1001
Cout 1
0 ns 5 ns 10 ns 15 ns 20 ns 25 ns
1000 1111
1001 1010
0001 1001

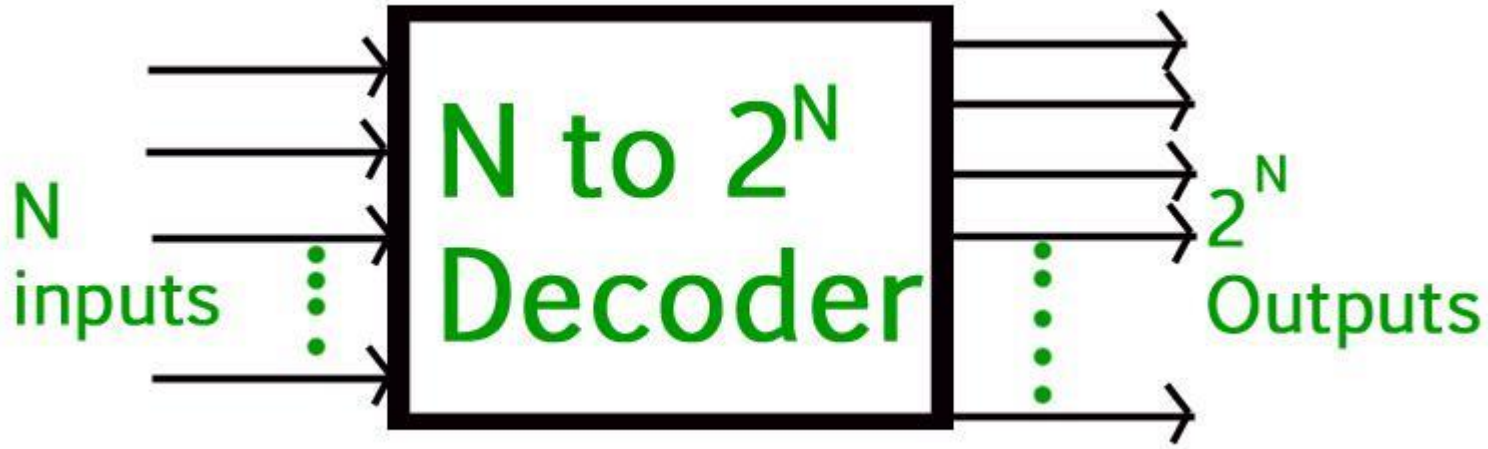
Schematic


ry_adder/Ripple_carry_adder.xpr] - Vivado 2014.4
Help
Default Layout
Ready
Elaborated Design - xc7vx485tffg1157-1 (active)
RTL Netlist
rca
Nets (17)
Leaf Cells (1)
FA1 (fulladder)
FA2 (fulladder)
FA3 (fulladder)
FA4 (fulladder)
Sources
RTL Netlist
Properties
Project Summary
RTL Schematic
rca.v
11 Cells
13 I/O Ports
27 Nets
A[3:0]
B[3:0]
FA1
FA2
FA3
FA4
fulladder
sum[3:0]
Cout

Schematic


carry_adder/Ripple_carry_adder.xpr] - Vivado 2014.4
Help
Default Layout
Ready
Elaborated Design - xc7vx485tffg1157-1 (active)
RTL Netlist
tb
Nets (2)
Leaf Cells (2)
A1 (rca)
Sources
RTL Netlist
Source File Properties
Project Summary
RTL Schematic
rca.v
33 Cells
59 Nets
A1
FA1
FA2
FA3
FA4
fulladder
RTL_AND
RTL_OR
RTL_XOR
rca

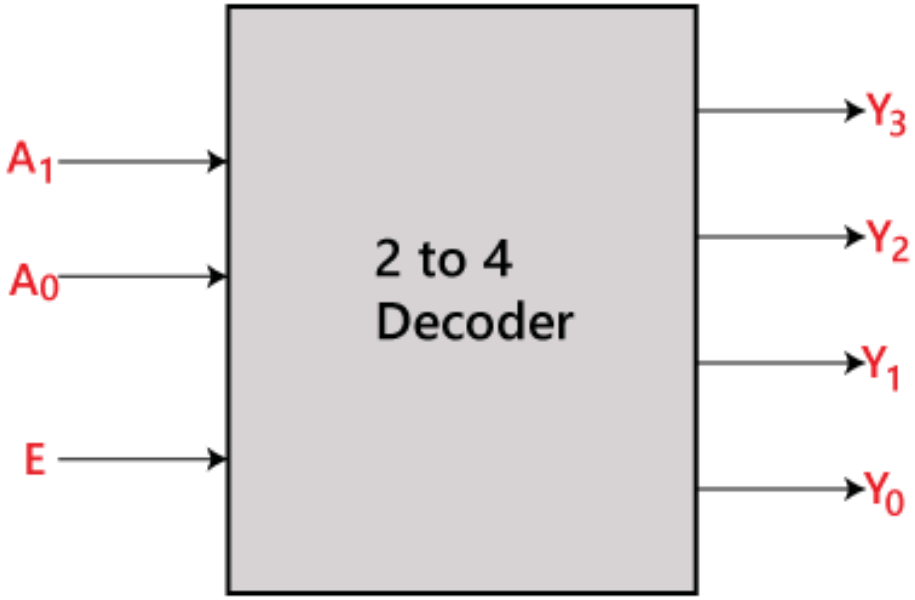
4. Decoder



- Decoder
- This Decoder is a combinational logic circuit, and its purpose is to decode the data given to it. It is made of n number of input lines and 2^n output lines. For every probable input condition, there are various output signals and depending on the input.
- Need of Decoder: Memory Addressing, Seven-Segment Displays, Control Systems

4. Decoder - 2 to 4 line decoder:

- In the 2 to 4 line decoder, there is a total of three inputs, i.e., A0, and A1 and E and four outputs, i.e., Y0, Y1, Y2, and Y3.
- For each combination of inputs, when the enable 'E' is set to 1, one of these four outputs will be 1.



Truth Table:

Enable	INPUTS		OUTPUTS			
E	A_1	A_0	Y_3	Y_2	Y_1	Y_0
0	X	X	0	0	0	0
1	0	0	0	0	0	1
1	0	1	0	0	1	0
1	1	0	0	1	0	0
1	1	1	1	0	0	0

4. Decoder - 2 to 4 line decoder:

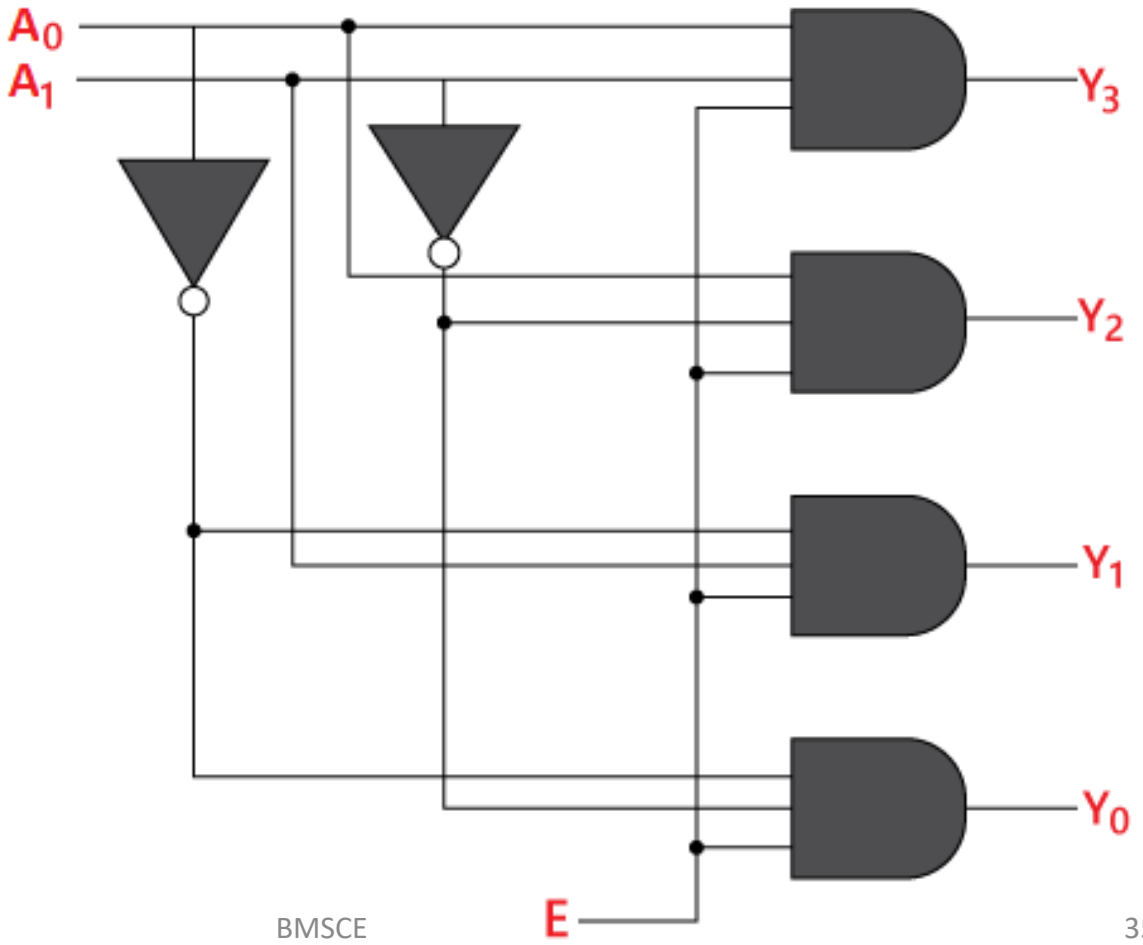
- The logical expression and circuit of the term Y0, Y0, Y2, and Y3 is as follows:

$$Y_0 = E \cdot \bar{A} \cdot \bar{B}$$

$$Y_1 = E \cdot \bar{A} \cdot B$$

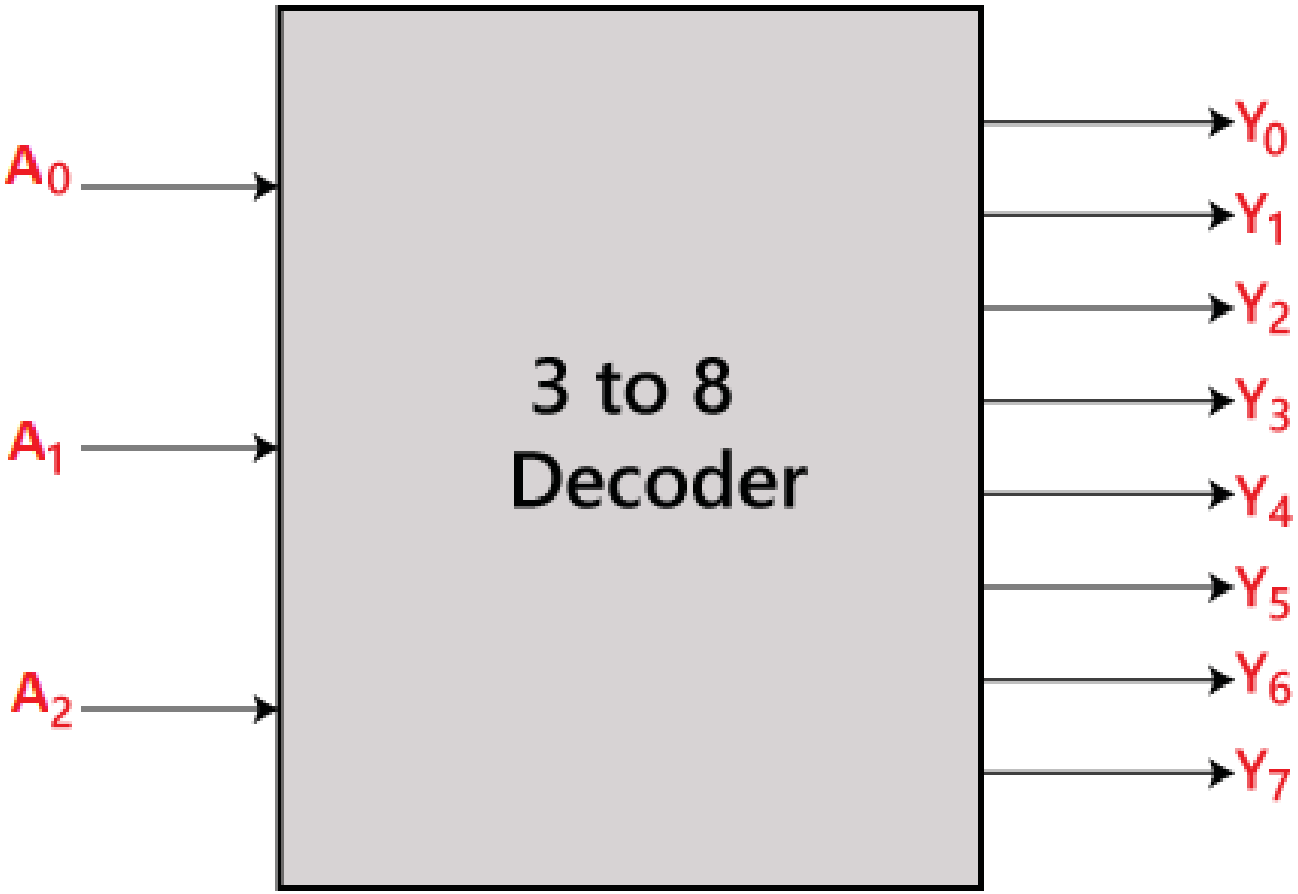
$$Y_2 = E \cdot A \cdot \bar{B}$$

$$Y_3 = E \cdot A \cdot B$$



4. Decoder - **3 to 8 line decoder:**

- The 3 to 8 line decoder is also known as **Binary to Octal Decoder.**
- In a 3 to 8 line decoder, there is a total of **8 outputs**, i.e., **Y0, Y1, Y2, Y3, Y4, Y5, Y6, and Y7** and **3 inputs, i.e., A0, A1, and A2**.
- This circuit has an enable input 'E'.
- Just like 2 to 4 line decoder, when enable 'E' is set to 1, one of these 8 outputs will be 1.
- The block diagram and the truth table of the 3 to 8 line encoder are given below.



EXP-4. Gate-level/Dataflow modelling: Decoders

3 to 8 line decoder:

Truth Table:

Enable	INPUTS			Outputs							
E	A_2	A_1	A_0	Y_7	Y_6	Y_5	Y_4	Y_3	Y_2	Y_1	Y_0
0	x	x	x	0	0	0	0	0	0	0	0
1	0	0	0	0	0	0	0	0	0	0	1
1	0	0	1	0	0	0	0	0	0	1	0
1	0	1	0	0	0	0	0	0	1	0	0
1	0	1	1	0	0	0	0	1	0	0	0
1	1	0	0	0	0	0	1	0	0	0	0
1	1	0	1	0	0	1	0	0	0	0	0
1	1	1	0	0	1	0	0	0	0	0	0
1	1	1	1	1	0	0	0	0	0	0	0

The logical expression of the term Y_0, Y_1, Y_2, Y_3, Y_4, Y_5, Y_6, and Y_7 is as follows:

$Y_0 = A_0'.A_1'.A_2'$

$Y_1 = A_0.A_1'.A_2'$

$Y_2 = A_0'.A_1.A_2'$

$Y_3 = A_0.A_1.A_2'$

$Y_4 = A_0'.A_1'.A_2$

$Y_5 = A_0.A_1'.A_2$

$Y_6 = A_0'.A_1.A_2$

$Y_7 = A_0.A_1.A_2$

EXP – 4: **Decoder –** 3 to 8 line decoder:

- Logical circuit of the above expressions is given below:

$Y_0 = A_0'.A_1'.A_2'$

$Y_1 = A_0.A_1'.A_2'$

$Y_2 = A_0'.A_1.A_2'$

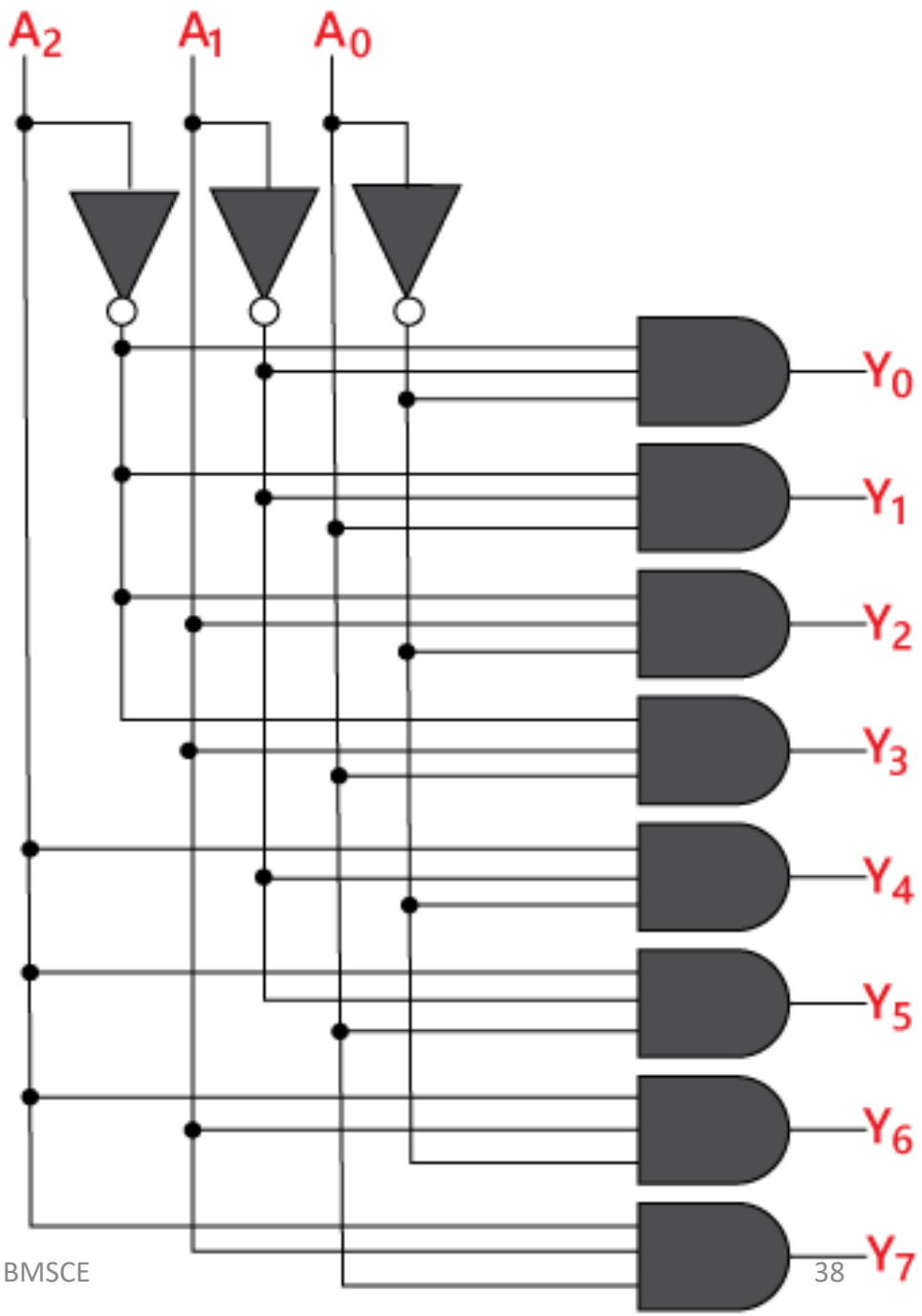
$Y_3 = A_0.A_1.A_2'$

$Y_4 = A_0'.A_1'.A_2$

$Y_5 = A_0.A_1'.A_2$

$Y_6 = A_0'.A_1.A_2$

$Y_7 = A_0.A_1.A_2$



a) Structural modelling (**gate level**): Decoder

```
module decoder( input E,a2,a1,a0, output
[7:0]Y );
not(a2n,a2);
not(a1n,a1);
not(a0n,a0);
and(Y[0],a2n,a1n,a0n,E);
and(Y[1],a2n,a1n,a0,E);
and(Y[2],a2n,a1,a0n,E);
and(Y[3],a2n,a1,a0,E);
and(Y[4],a2,a1n,a0n,E);
and(Y[5],a2,a1n,a0,E);
and(Y[6],a2,a1,a0n,E);
and(Y[7],a2,a1,a0,E);
endmodule
```

```
module tb;
wire [7:0]Y;
reg E,a2,a1,a0;
decoder A1(E,a2,a1,a0,Y);
initial begin
E=1;a2=0;a1=0;a0=0;
#5 E=1;a2=0;a1=0;a0=1;
#5 E=1;a2=0;a1=1;a0=0;
#5 E=1;a2=0;a1=1;a0=1;
#5 E=1;a2=1;a1=0;a0=0;
#5 E=1;a2=1;a1=0;a0=1;
#5 E=1;a2=1;a1=1;a0=0;
#5 E=1;a2=1;a1=1;a0=1;
#15 $finish;
end
endmodule
```

b) Data Flow modelling: Decoder

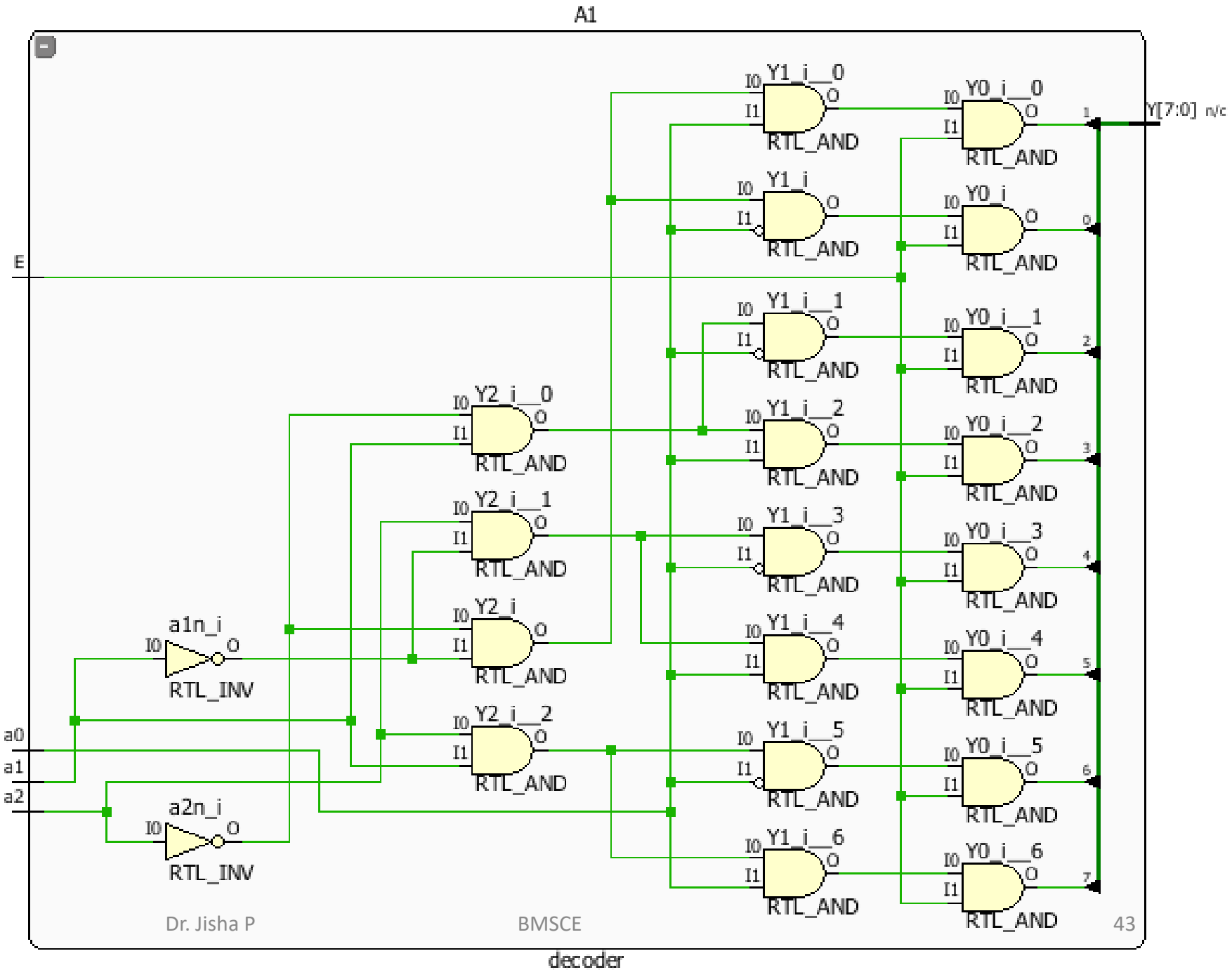
```
module decoder( input E,a2,a1,a0, output [7:0]Y);
assign Y[0]=((~a2)&(~a1)&(~a0)&E);
assign Y[1]=((~a2)&(~a1)&(a0)&E);
assign Y[2]=((~a2)&(a1)&(~a0)&E);
assign Y[3]=((~a2)&(a1)&(a0)&E);
assign Y[4]=((a2)&(~a1)&(~a0)&E);
assign Y[5]=((a2)&(~a1)&(a0)&E);
assign Y[6]=((a2)&(a1)&(~a0)&E);
assign Y[7]=((a2)&(a1)&(a0)&E);
endmodule
```

c) Behavioral Modelling: Decoder

```
module decoder( input E,a2,a1,a0, output reg [7:0]Y);
always@(E,a2,a1,a0)
if(E==1)
case({a2,a1,a0})
3'b000:Y=8'd1;
3'b001:Y=8'd2;
3'b010:Y=8'd4;
3'b011:Y=8'd8;
3'b100:Y=8'd16;
3'b101:Y=8'd32;
3'b110:Y=8'd64;
3'b111:Y=8'd128;
default:Y=1'b0;
endcase
endmodule
```

Simulation Output

Schematic


A1
E
a0
a1
a2
a1n_i
a2n_i
RTL_INV
Y2_i__0
Y2_i__1
Y2_i
Y2_i__2
Y1_i__0
Y1_i
Y1_i__1
Y1_i__2
Y1_i__3
Y1_i__4
Y1_i__5
Y1_i__6
Y0_i__0
Y0_i
Y0_i__1
Y0_i__2
Y0_i__3
Y0_i__4
Y0_i__5
Y0_i__6
RTL_AND
Y[7:0] n/c
decoder

EXP-5: Dataflow modelling for 2-bit magnitude comparator

- Magnitude Comparator
- Magnitude comparator is a type of Combinational circuit, it Basically compares two binary numbers and determines their relative magnitude.
- It gives output whether one number is greater than the other, or less than or equal.
- These comparators are used in digital systems, such as for sorting networks, and decision-making circuits to handle numerical comparisons perfectly without any error.

Magnitude Comparator

- The circuit works by comparing the bits of the two numbers starting from the most significant bit (MSB) and moving toward the least significant bit (LSB). At each bit position, the two corresponding bits of the numbers are compared. If the bit in the first number is greater than the corresponding bit in the second number, the A>B output is set to 1,and the circuit immediately determines that the first number is greater than the second. Similarly, if the bit in the second number is greater than the corresponding bit in the first number, the A<B output is set to 1, and the circuit immediately determines that the first number is less than the second.

- If the two corresponding bits are equal, the circuit moves to the next bit position and compares the next pair of bits. This process continues until all the bits have been compared. If at any point in the comparison, the circuit determines that the first number is greater or less than the second number, the comparison is terminated, and the appropriate output is generated.

- If all the bits are equal, the circuit generates an A=B output, indicating that the two numbers are equal.

- There are different ways to implement a magnitude comparator, such as using a combination of XOR, AND, and OR gates, or by using a cascaded arrangement of full adders. The choice of implementation depends on factors such as speed, complexity, and power consumption.

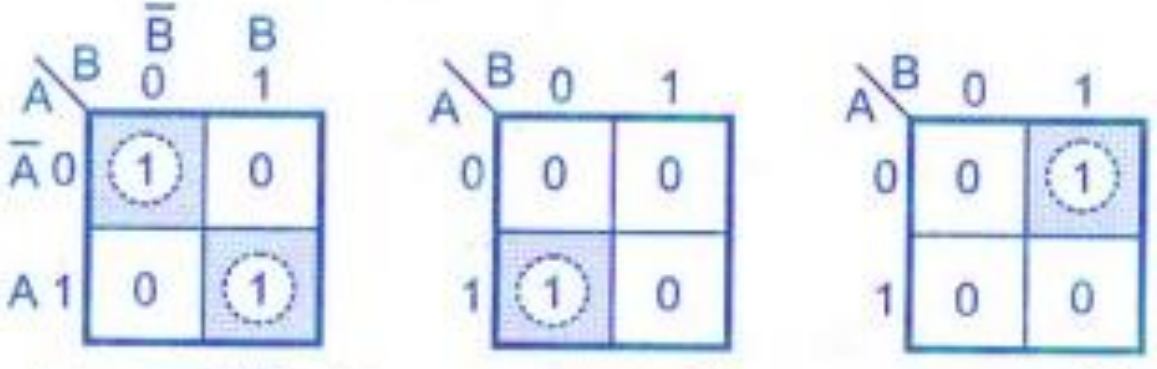
1-Bit Magnitude Comparator

- A comparator used to compare two bits is called a single-bit comparator.
- It consists of two inputs each for two single-bit numbers and three outputs to generate less than, equal to, and greater than between two binary numbers.
- The truth table for a 1-bit comparator is given.

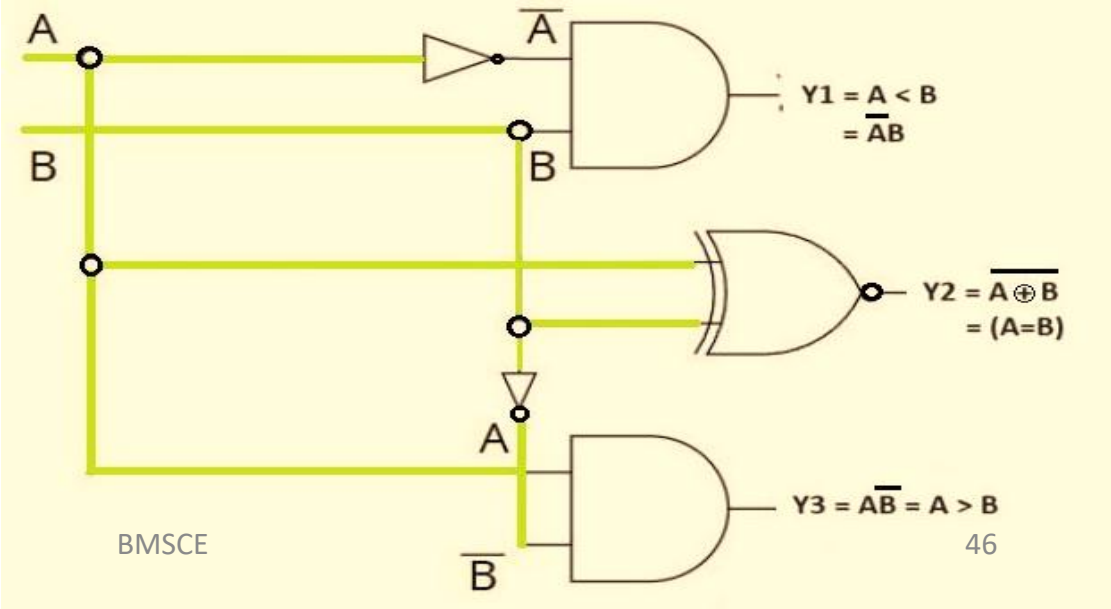
A	B	A<B	A=B	A>B
0	0	0	1	0
0	1	1	0	0
1	0	0	0	1
1	1	0	1	0

From the above truth table logical expressions for each output can be expressed as follows.

A>B: AB' A<B: A'B A=B: A'B' + AB



(a) k-map simplification



EXP-5:. Data Flow modelling:2bit comparator

```
module comparator_1bit(     input a, b,    output aeqb, altb, agtb );

   // Dataflow modeling
   assign aeqb = (a ~^ b);          // a equal to b
   assign altb = (~a) & b;          // a less than b
   assign agtb = a & (~b);          // a greater than b

endmodule
```

EXP-5:. Data Flow modelling:2bit comparator

```
module tb;
wire aeqb,altb,agtb;
reg b1, b0, a1, a0;
comparator A1(b1,b0,a1,a0 ,aeqb,altb,agtb);
initial
begin
a=0;b=1;
#10 a=1;b=0;
#10 a=1;b=1;
#20 $finish;
end
endmodule
```

2-Bit Magnitude Comparator

- A comparator used to compare two binary numbers each of two bits is called a 2-bit Magnitude comparator. It consists of four inputs and three outputs to generate less than, equal to, and greater than between two binary numbers.

- The truth table for a 2-bit comparator is given.

INPUT				OUTPUT		
A1	**A0**	**B1**	**B0**	**A<B**	**A=B**	**A>B**
0	0	0	0	0	1	0
0	0	0	1	1	0	0
0	0	1	0	1	0	0
0	0	1	1	1	0	0
0	1	0	0	0	0	1
0	1	0	1	0	1	0
0	1	1	0	1	0	0
0	1	1	1	1	0	0
1	0	0	0	0	0	1
1	0	0	1	0	0	1
1	0	1	0	0	1	0
1	0	1	1	1	0	0
1	1	0	0	0	0	1
1	1	0	1	0	0	1
1	1	1	0	0	0	1
1	1	1	1	0	1	0

2-Bit Magnitude Comparator

A > B

A_1A_0 \ B_1B_0	00	01	11	10
00	0	0	0	0
01	1	0	0	0
11	1	1	0	1
10	1	1	0	0

A = B

A_1A_0 \ B_1B_0	00	01	11	10
00	1	0	0	0
01	0	1	0	0
11	0	0	1	0
10	0	0	0	1

A < B

A_1A_0 \ B_1B_0	00	01	11	10
00	0	1	1	1
01	0	0	1	1
11	0	0	0	0
10	0	0	1	0

$A>B = A_0\overline{B}_1\overline{B}_0 + A_1\overline{B}_1 + A_1A_0\overline{B}_0$

Fig. 1.14.2

$$(A = B) = \overline{A}_1\,\overline{A}_0\,\overline{B}_1\overline{B}_0 + \overline{A}_1A_0\,\overline{B}_1B_0 + A_1A_0\,B_1B_0 + A_1\overline{A}_0\,B_1\overline{B}_0$$

$$= \overline{A}_1\overline{B}_1\,(\overline{A}_0\overline{B}_0 + A_0B_0) + A_1B_1\,(A_0B_0 + \overline{A}_0\overline{B}_0)$$

$$= (A_0 \odot B_0)\,(A_1 \odot B_1)$$

$$(A < B) = \overline{A}_1\overline{A}_0\,B_0 + \overline{A}_0\,B_1B_0 + \overline{A}_1B_1$$

2-Bit Magnitude Comparator

- Logical expressions for each output can be expressed as follows

$$A>B:\ G = A0\,\overline{B1}\ \overline{B0} + A1\,\overline{B1} + A1\ A0\,\overline{B0}$$

$$A=B:\ E = \overline{A1}\ \overline{A0}\,\overline{B1}\ \overline{B0} + \overline{A1}\,A0\ \overline{B1}\,B0 + A1\ A0\,B1\,B0 + A1\,\overline{A0}\,B1\,\overline{B0}$$

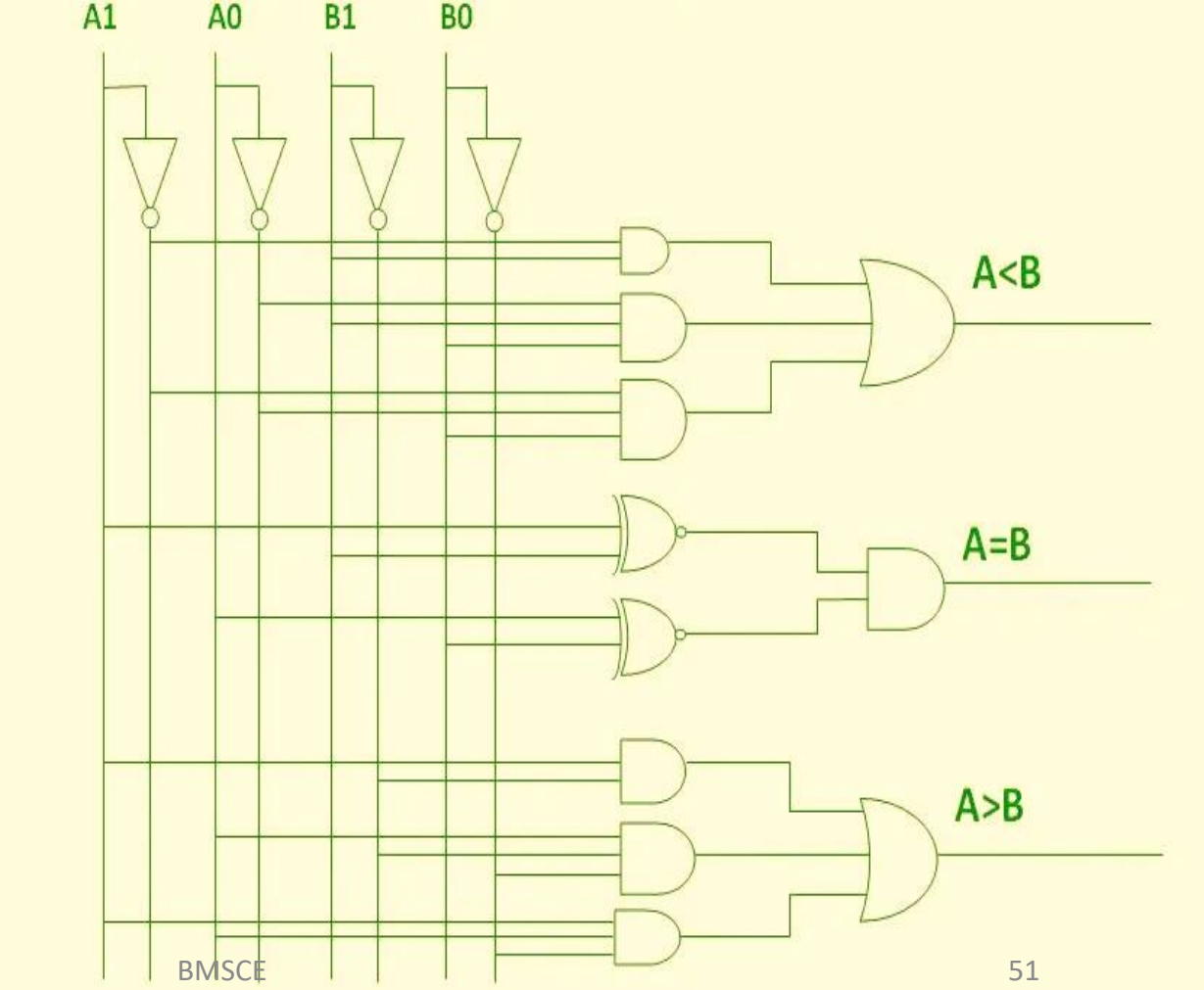
$$= \overline{A1}\ \overline{B1}\,(\overline{A0}\ \overline{B0} + A0\,B0) + A1\,B1\,(A0\,B0 + \overline{A0}\ \overline{B0})$$

$$= (A0\,B0 + \overline{A0}\ \overline{B0})\,(A1\,B1 + \overline{A1}\ \overline{B1})$$

$$= (A0 \text{ Ex-NOR } B0)\,(A1 \text{ Ex-NOR } B1)$$

$$A<B:\ L = \overline{A1}\ B1 + \overline{A0}\,B1\,B0 + \overline{A1}\ \overline{A0}\,B0$$

Logic circuit for this comparator as given below.



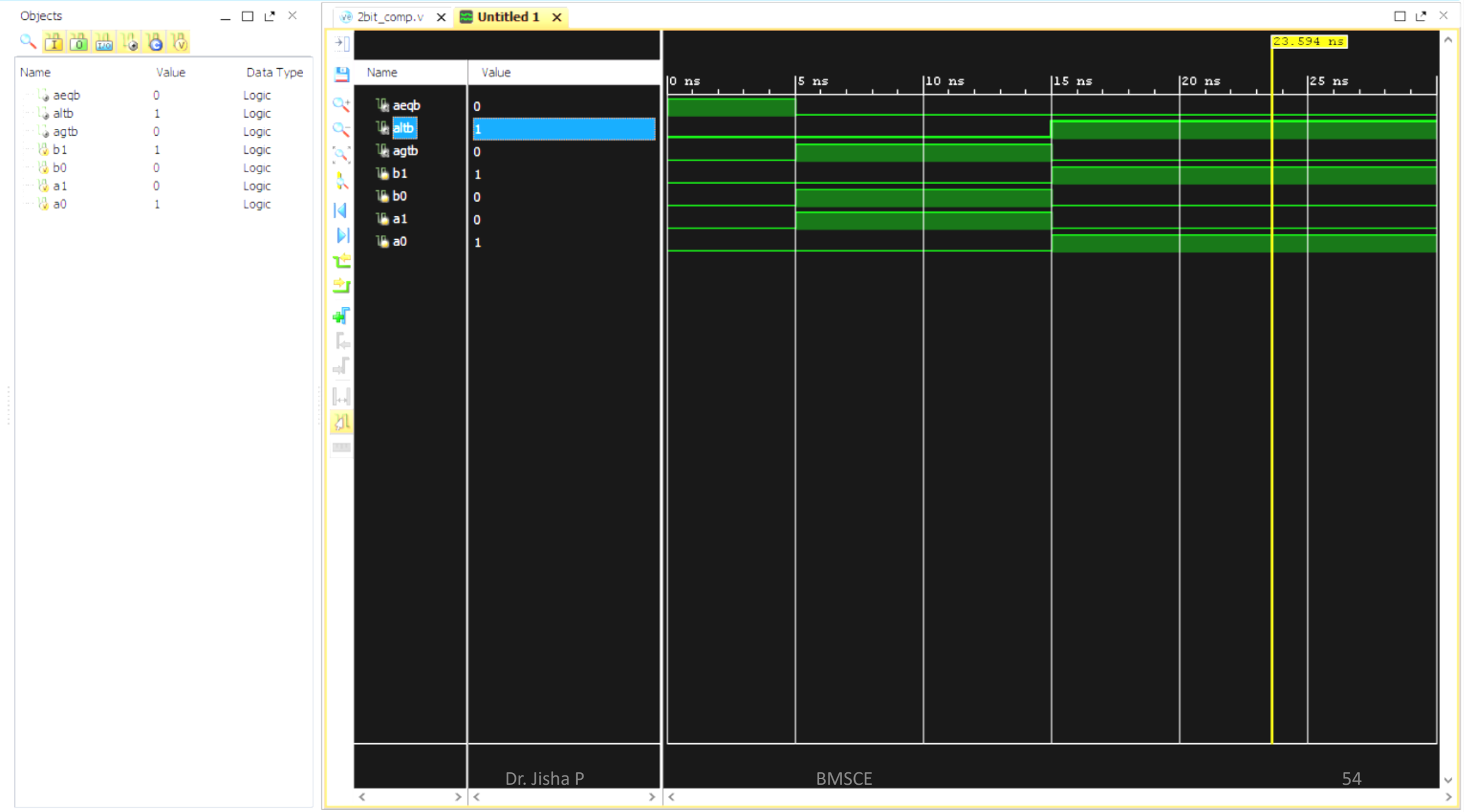
EXP-5:. Data Flow modelling:2bit comparator

```
module comparator( input b1,b0,a1,a0,output aeqb, altb, agtb) ;

assign altb = ((b1)&(~a1))|((b0)&(~a1)&(~a0))|((b1)&(b0)&(~a0));

assign agtb = ((a1)&(~b1))|((a1)&(a0)&(~b0))|((a0)&(~b1)&(~b0));

assign aeqb = (~(altb|agtb));

endmodule
```

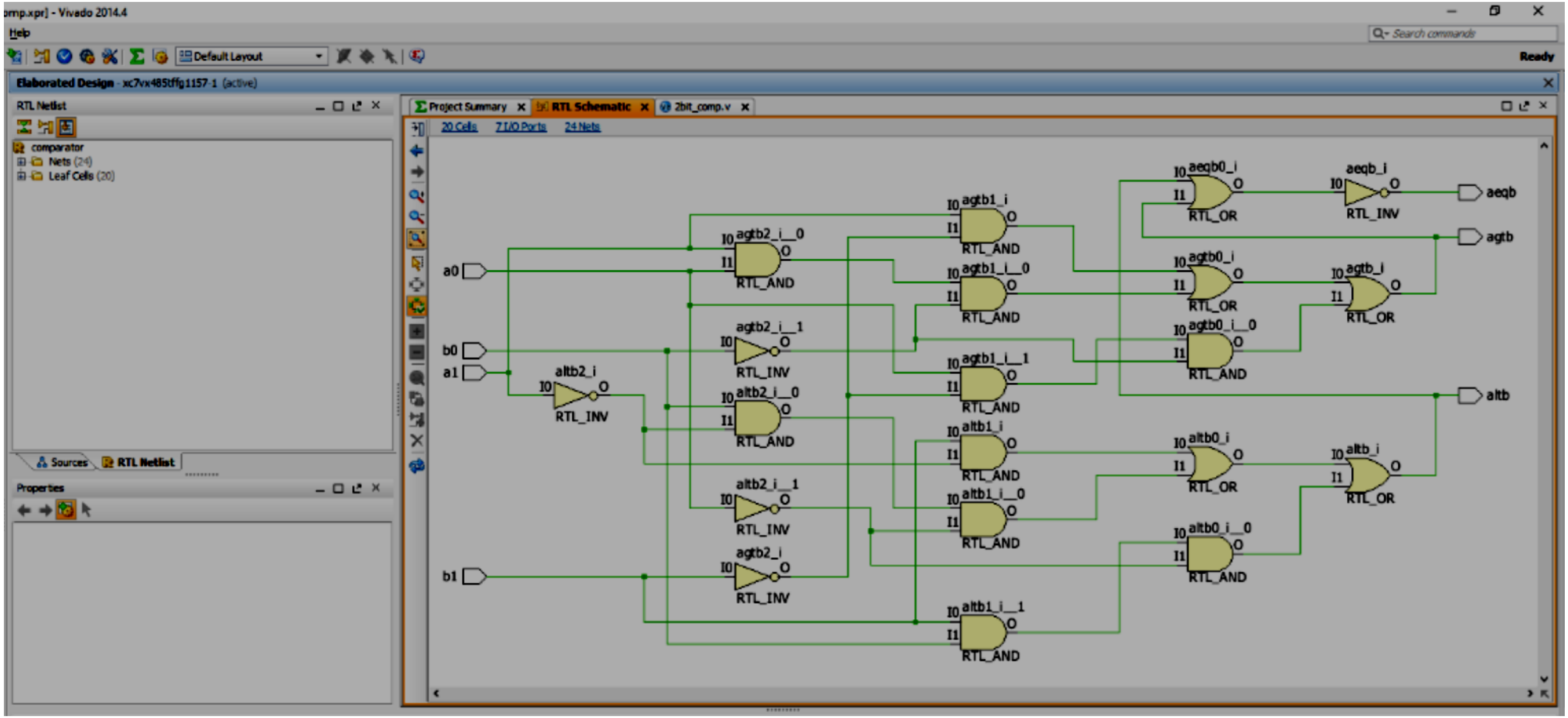
EXP-5:. Data Flow modelling:2bit comparator

```
module tb;
wire aeqb,altb,agtb;
reg b1, b0, a1, a0;
comparator A1(b1,b0,a1,a0 ,aeqb,altb,agtb);
initial
begin
a1 = 0; a0 = 0; b1 = 0; b0 = 0; #10;   // A=B
a1 = 0; a0 = 0; b1 = 0; b0 = 1; #10; // A < B
a1 = 0; a0 = 1; b1 = 0; b0 = 0; #10; // A > B
a1 = 0; a0 = 1; b1 = 0; b0 = 1; #10; // A=B
..
..
$finish;
end
endmodule
```

Simulation Output

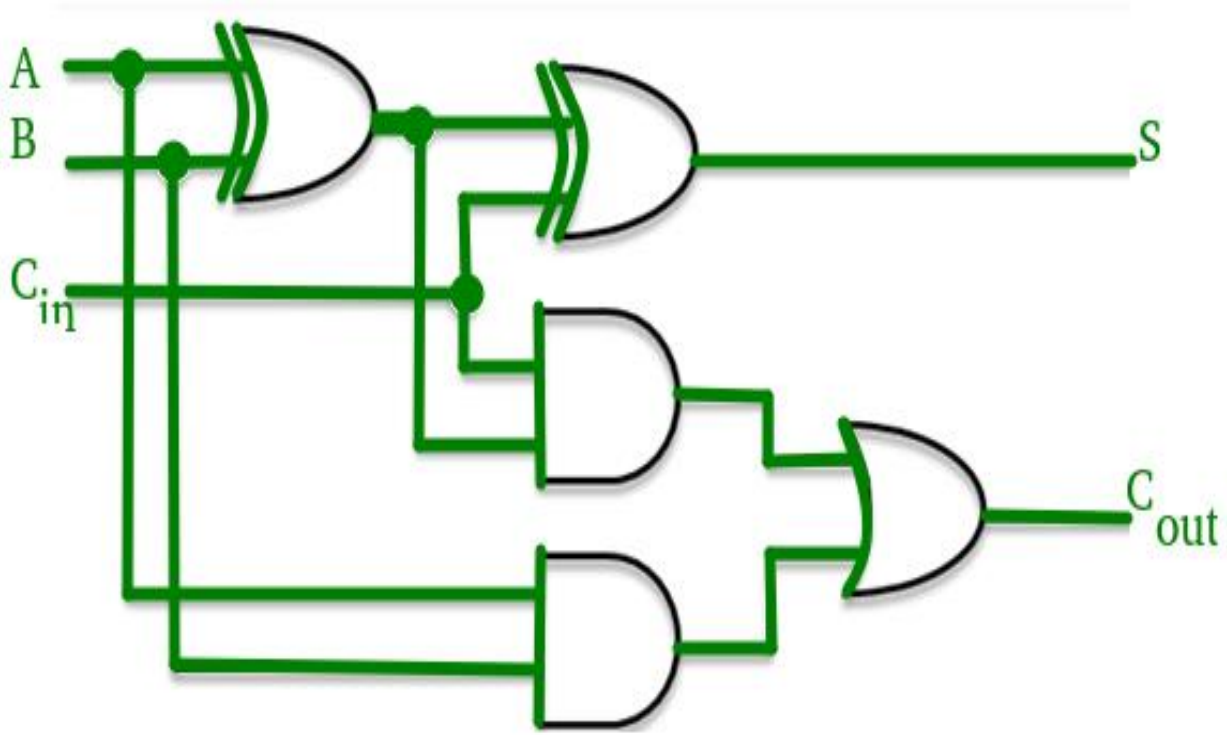


Schematic


Vivado 2014.4
Help
Default Layout
Ready
Elaborated Design - xc7vx485tffg1157-1 (active)
RTL Netlist
comparator
Nets (24)
Leaf Cells (20)
Sources
RTL Netlist
Properties
Project Summary
RTL Schematic
2bit_comp.v
20 Cells
7 I/O Ports
24 Nets
a0
b0
a1
b1
altb2_i
RTL_INV
agtb2_i__0
RTL_AND
agtb2_i__1
RTL_INV
altb2_i__0
RTL_AND
altb2_i__1
RTL_INV
agtb2_i
RTL_INV
agtb1_i
RTL_AND
agtb1_i__0
RTL_AND
agtb1_i__1
RTL_AND
altb1_i
RTL_AND
altb1_i__0
RTL_AND
altb1_i__1
RTL_AND
aeqb0_i
RTL_OR
aeqb_i
RTL_INV
agtb0_i
RTL_OR
agtb_i
RTL_OR
agtb0_i__0
RTL_AND
altb0_i
RTL_OR
altb_i
RTL_OR
altb0_i__0
RTL_AND
aeqb
agtb
altb

6. Data Flow Modelling - Carry Look Ahead Adder

- Carry Look-ahead Adder :
- A carry look-ahead adder reduces the propagation delay by introducing more complex hardware.
- In this design, the ripple carry design is suitably transformed such that the carry logic over fixed groups of bits of the adder is reduced to two-level logic.



A	B	C	C +1	Condition
0	0	0	0	No Carry Generate
0	0	1	0	
0	1	0	0	
0	1	1	1	Carry Propogate
1	0	0	0	
1	0	1	1	
1	1	0	1	Carry Generate
1	1	1	1	

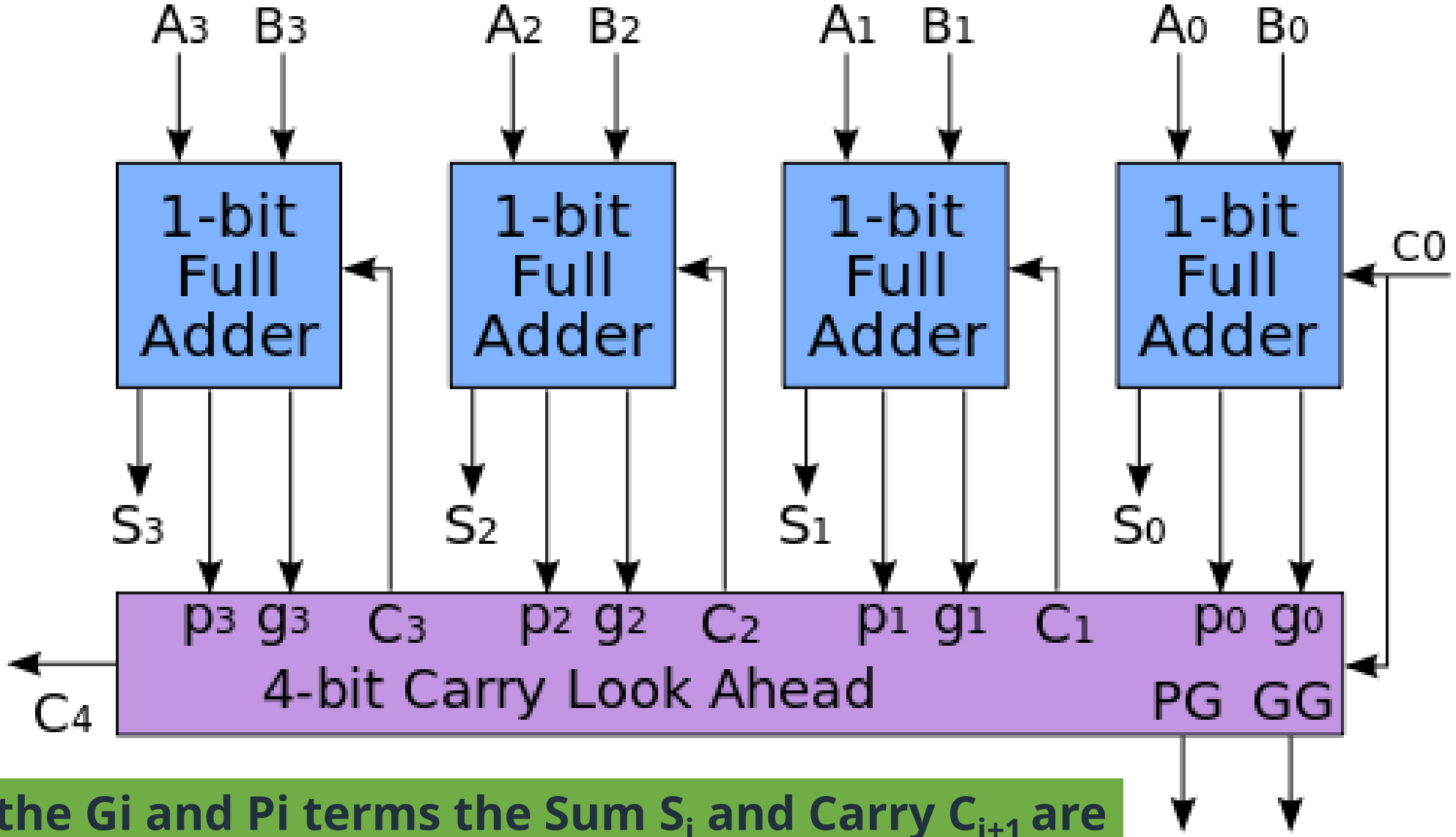
Consider the full adder circuit shown above with corresponding truth table. We define two variables as 'carry generate' $\mathbf{G_i}$ and 'carry propagate' $\mathbf{P_i}$ then,

$\mathbf{G_i = A_iB_i}$

where G is called carry generator

$\mathbf{P_i = A_i \oplus B_i}$

where P is called carry propagator



Using the Gi and Pi terms the Sum S_i and Carry C_{i+1} are given as below –

$S_i = P_i \oplus G_i$

$C_{i+1} = C_i . P_i + G_i$

Then, re-writing the above equations, we have-

$$C_1 = C_0P_0 + G_0 \ldots\ldots\ldots\ldots\ldots (1)$$
$$C_2 = C_1P_1 + G_1 \ldots\ldots\ldots\ldots\ldots (2)$$
$$C_3 = C_2P_2 + G_2 \ldots\ldots\ldots\ldots\ldots (3)$$
$$C_4 = C_3P_3 + G_3 \ldots\ldots\ldots\ldots\ldots (4)$$

- **Clearly, C1, C2 and C3 are intermediate carry bits.**
- **So, let's remove C_1, C_2 and C_3 from RHS of every equation.**
- **Substituting (1) in (2) we get C_2 in terms of**
- **Then, substituting (2) (3), we get C_3 in terms C_0 and so on.**

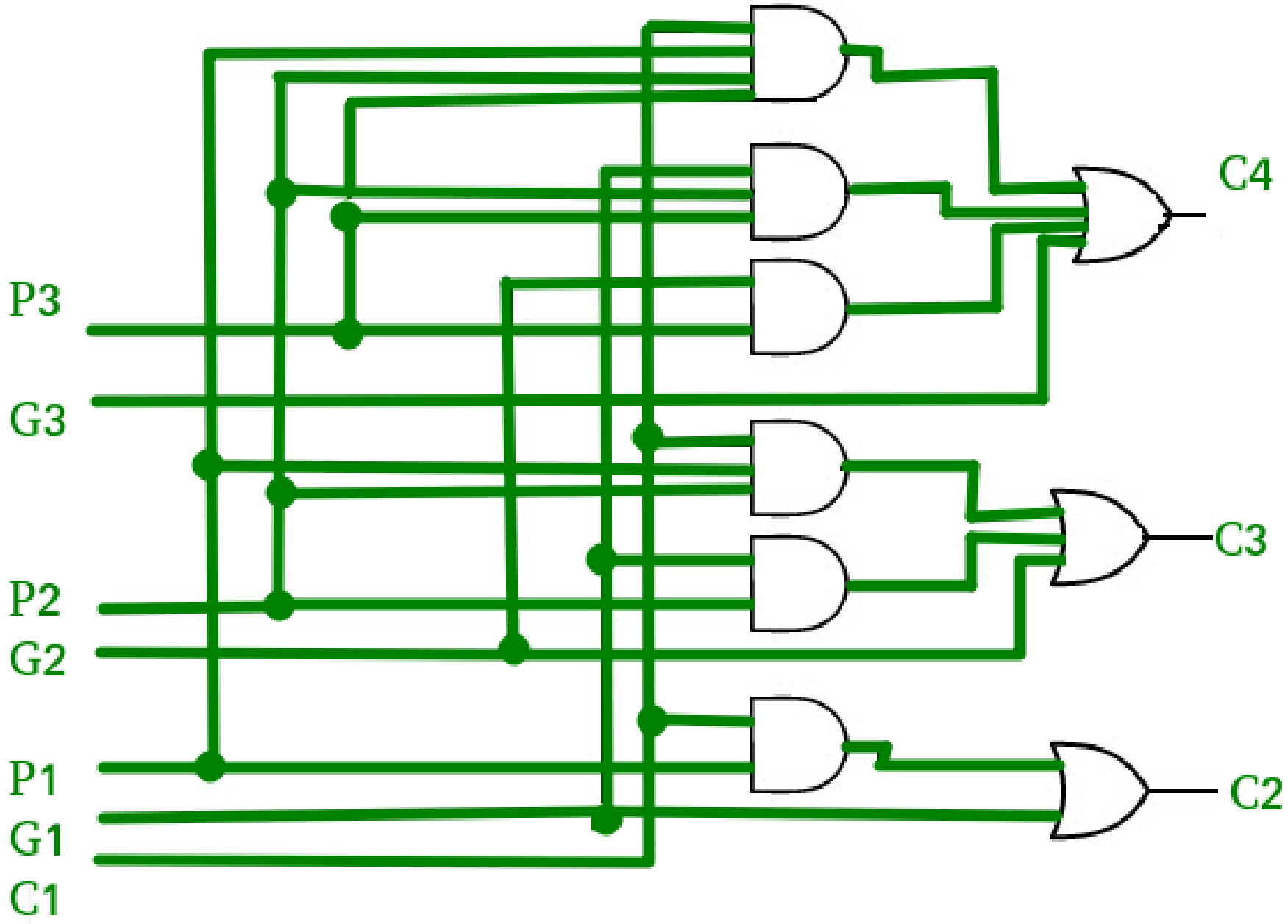
Finally, we have the following equations-

$$C_1 = C_0P_0 + G_0$$

$$C_2 = C_0P_0P_1 + G_0P_1 + G_1$$

$$C_3 = C_0P_0P_1P_2 + G_0P_1P_2 + G_1P_2 + G_2$$

$$C_{out} = C_4 = C_0P_0P_1P_2P3 + G_0P_1P_2P_3 + G_1P_2P_3 + G_2P_3 + G_3$$


C4
P3
G3
C3
P2
G2
P1
C2
G1
C1

6. Data Flow Modelling - Carry Look Ahead Adder

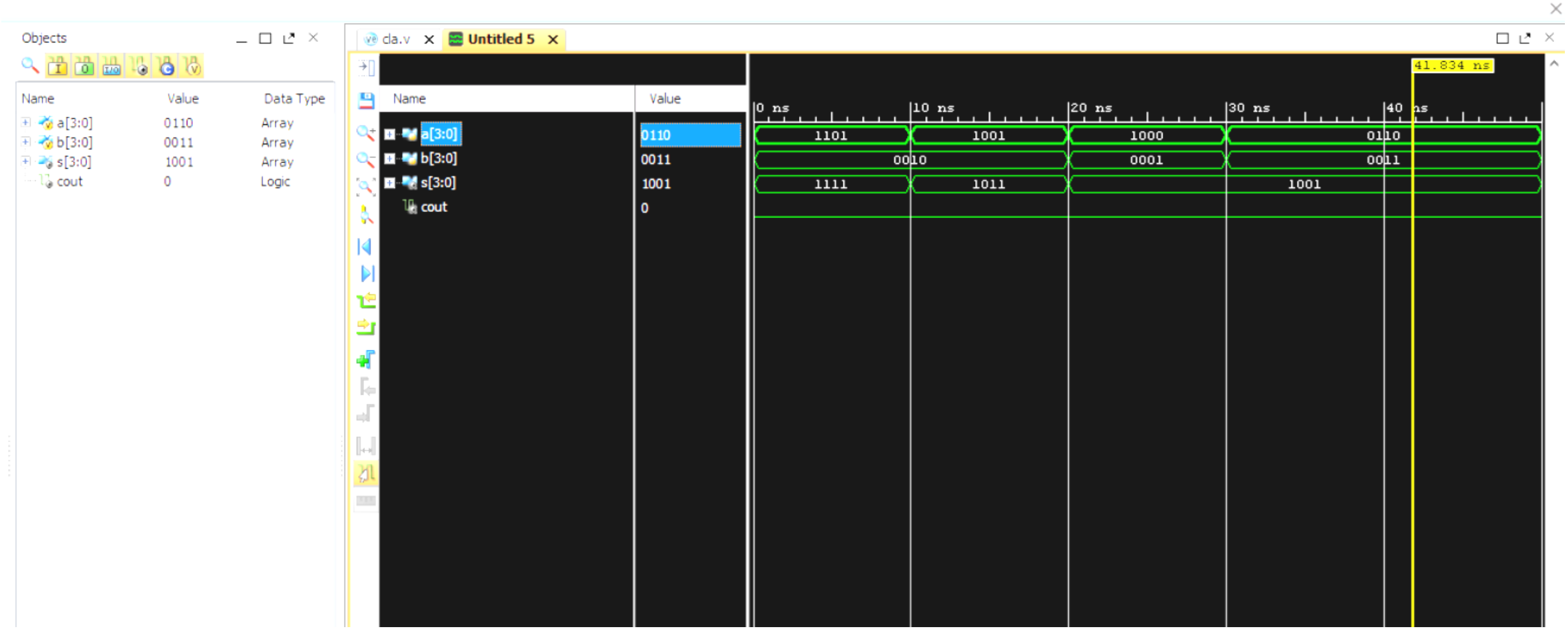
```
module cla( input [3:0]a,b,
output [3:0]s,output cout );

wire [3:0]g,p,c;
assign g[0]=a[0]&b[0];
assign g[1]=a[1]&b[1];
assign g[2]=a[2]&b[2];
assign g[3]=a[3]&b[3];
assign p[0]=a[0]^b[0];
assign p[1]=a[1]^b[1];
assign p[2]=a[2]^b[2];
assign p[3]=a[3]^b[3];
```

```
assign c[0]=0;
assign c[1]=g[0]|(p[0]&c[0]);
assign c[2]=g[1]|(p[1]&g[0])|(p[1]&p[0]&c[0]);
assign c[3]=g[2]|(p[2]&g[1])|(p[2]&p[1]&g[0])|
(p[2]&p[1]&p[0]&c[0]);
assign cout=g[3]|(p[3]&g[2])|(p[3]&p[2]&g[1])|
(p[3]&p[2]&p[1]&g[0])|(p[3]&p[2]&p[1]&p[0]&
c[0]);
assign s[0]=a[0]^b[0]^c[0];
assign s[1]=a[1]^b[1]^c[1];
assign s[2]=a[2]^b[2]^c[2];
assign s[3]=a[3]^b[3]^c[3];
endmodule
```

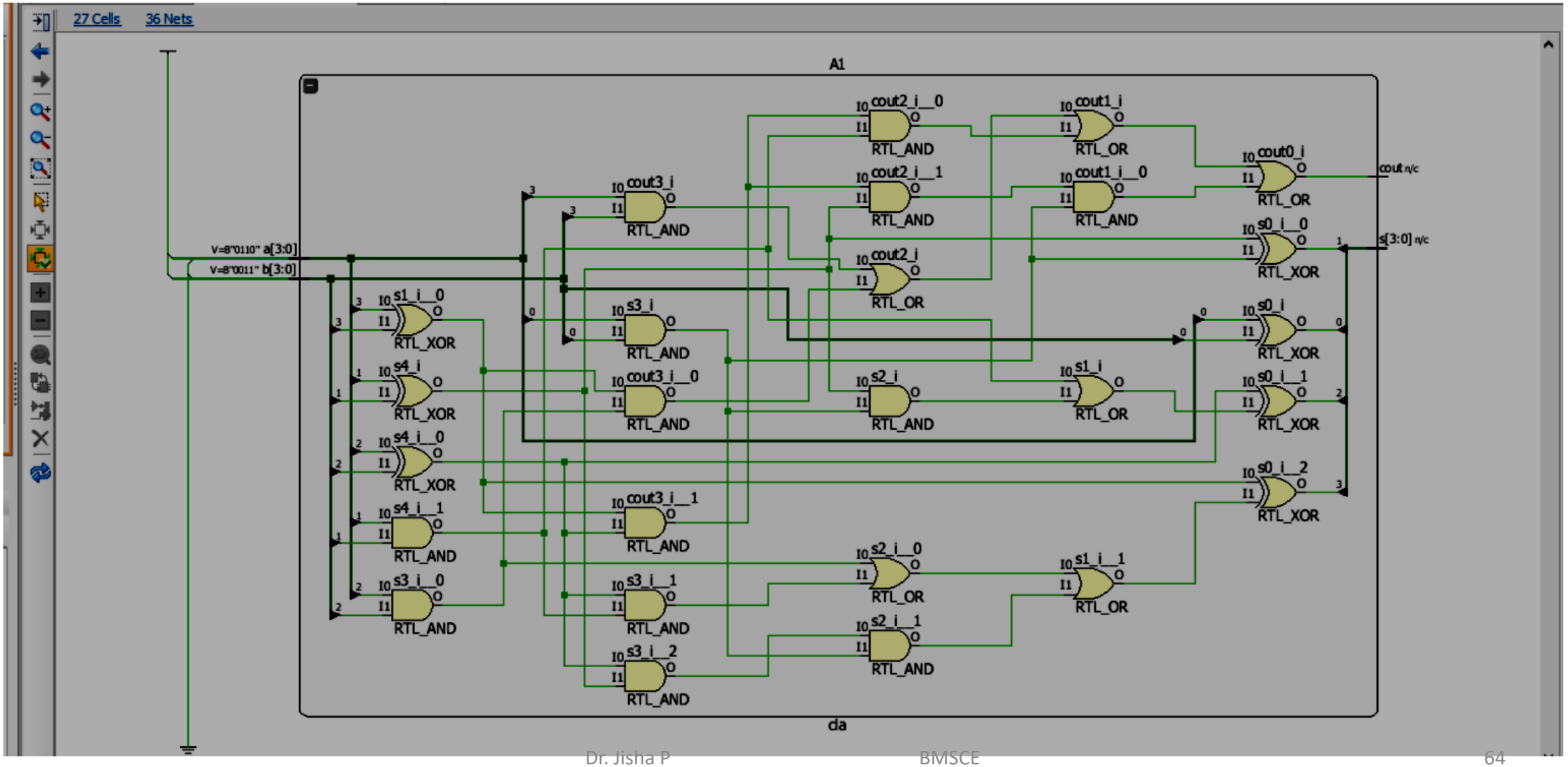
6. Data Flow Modelling - Carry Look Ahead Adder

```
module tb;
reg [3:0]a,b;
wire [3:0]s;
wire cout;
cla A1(a,b,s,cout);
initial
begin
a=4'b1101; b=4'b0010;
#10 a=4'd9; b=4'd2;
#10 a=4'd8; b=4'd1;
#10 a=4'd6; b=4'd3;
#20 $finish;
end
endmodule
```

Simulation Output



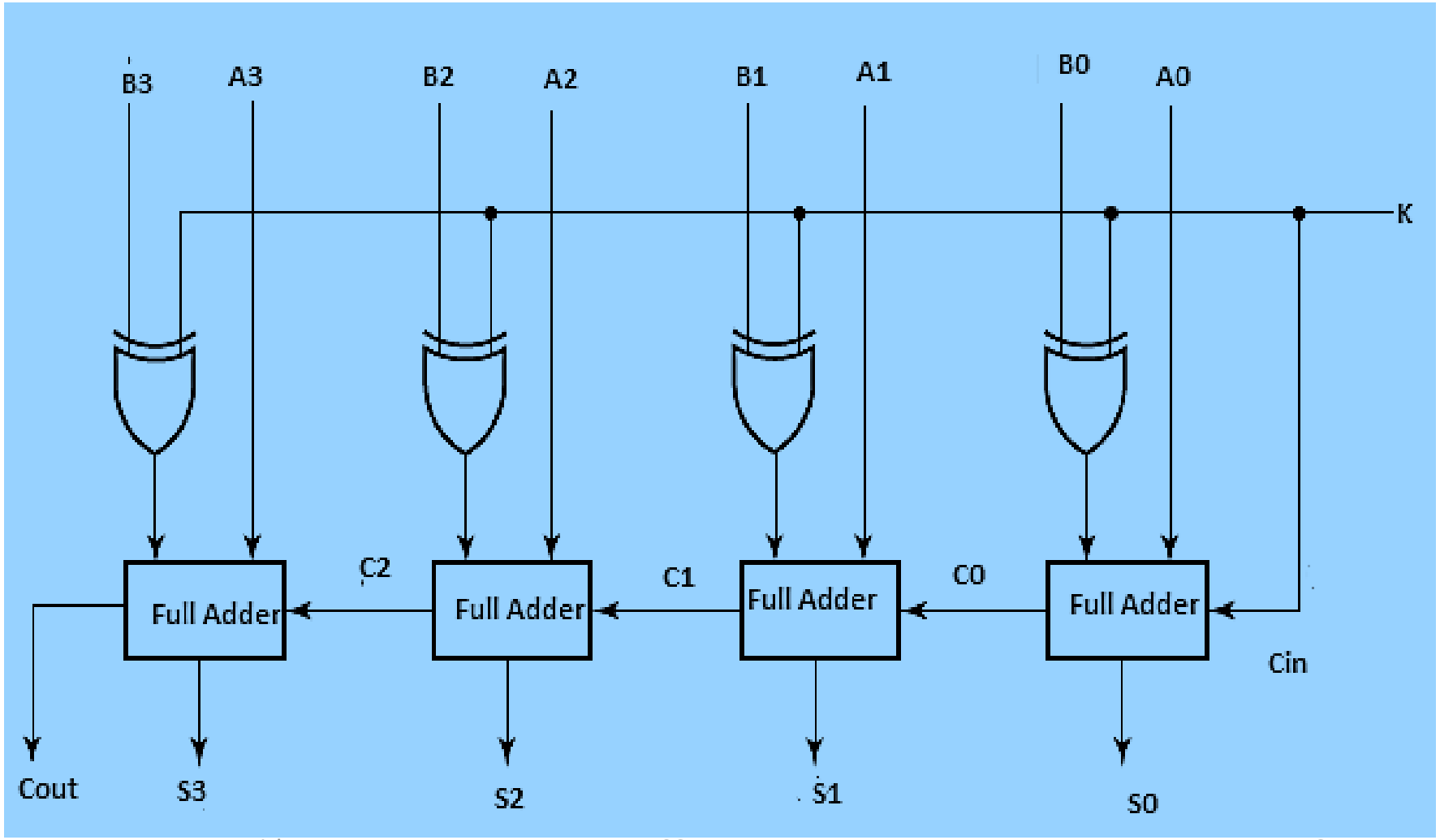
Schematic


27 Cells
36 Nets
A1
V=B"0110" a[3:0]
V=B"0011" b[3:0]
cout n/c
s[3:0] n/c
cout2_i__0
cout1_i
cout0_i
cout2_i__1
cout1_i__0
cout3_i
cout2_i
s0_i__0
s1_i__0
s3_i
s0_i
s4_i
cout3_i__0
s2_i
s1_i
s0_i__1
s4_i__0
s0_i__2
cout3_i__1
s4_i__1
s2_i__0
s1_i__1
s3_i__0
s3_i__1
s2_i__1
s3_i__2
RTL_AND
RTL_OR
RTL_XOR
cla

7. Structural Modelling: Multibit Subtractor (using Adder)

The module computes the subtraction $a-b$a−b by:

1. Calculating the 1's complement of b ($\sim b$).
2. Adding $a+\sim b+1$ using a chain of full adders.
3. The result is provided as s, with the final carry-out as cout.



Multibit Subtractor (using Adder)

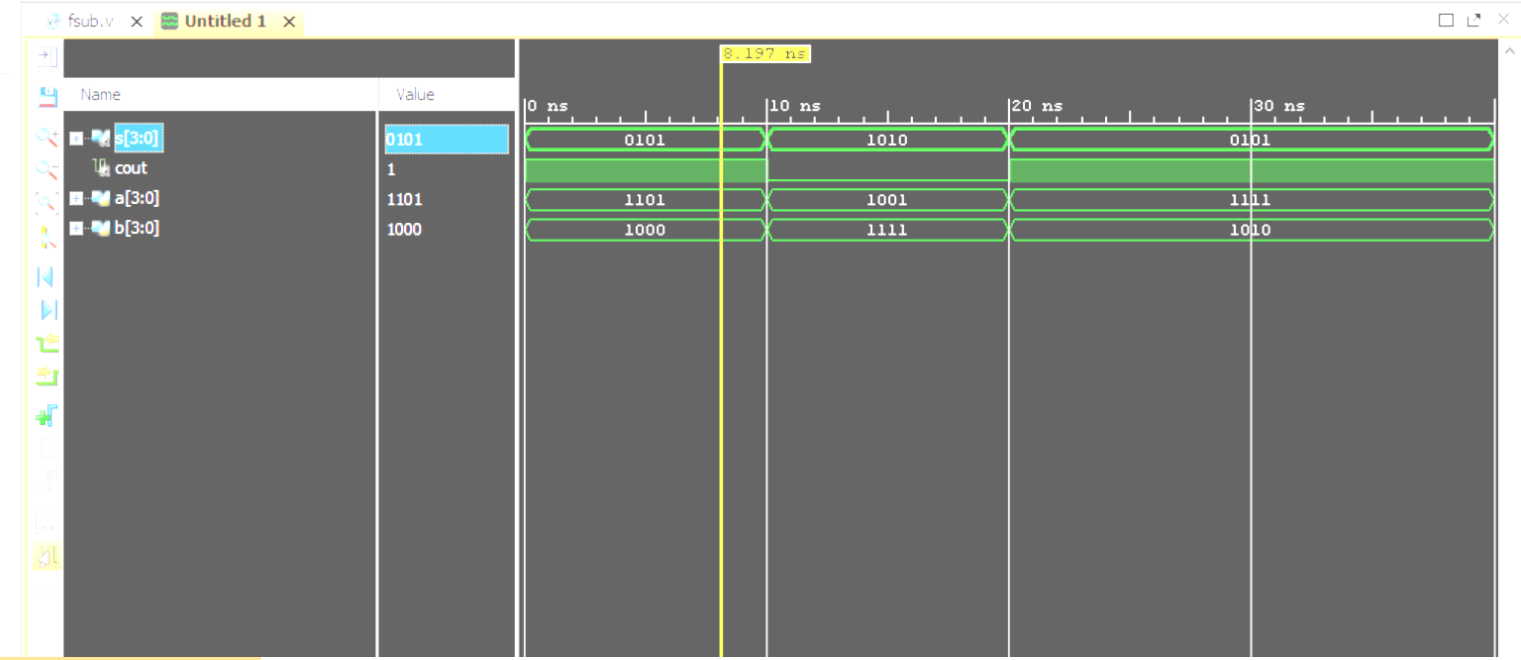
- In binary subtraction, a−b can be computed as: $a-b=a+(\sim b+1)$
- Here:$\sim b$: The bitwise NOT of b (1's complement).
- Adding 1 to $\sim b$ gives the two's complement of b, effectively converting the subtraction into an addition.
- assign c[0] = 1;This initializes the carry-in for the first stage to 1, which effectively adds the "+1" required for the two's complement.
- Full Adders: The module uses four instances of a full adder to perform the bitwise addition of a and the two's complement of b.
- Each full adder takes:**A bit of a (a[i]),A bit of y ($\sim b[i]$),The carry-in from the previous stage (c[i]).**
- The outputs are:A sum bit (s[i]).A carry-out for the next stage (c[i+1]).

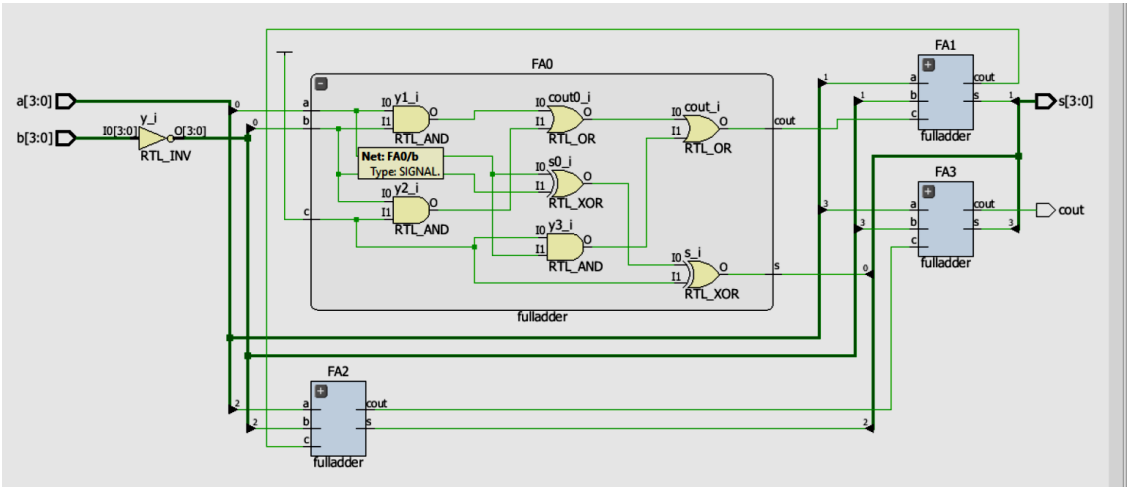
7. Full Subtractor

```
module fsub( input [3:0]a,b,
output [3:0]s, output cout );
wire [3:0]y;
assign y=~b;
wire [3:0]c;
assign c[0]=1;
fulladder FA0(a[0],y[0],c[0],s[0],c[1]);
fulladder FA1(a[1],y[1],c[1],s[1],c[2]);
fulladder FA2(a[2],y[2],c[2],s[2],c[3]);
fulladder FA3(a[3],y[3],c[3],s[3],cout);
endmodule
```

```
module fulladder( input a,b,c,
output s,cout  );
xor(s,a,b,c);
and(y1,a,b);
and(y2,b,c);
and(y3,c,a);
or(cout,y1,y2,y3);
endmodule
```

```
module tb;
wire [3:0]s;
wire cout;
reg [3:0]a,b;
fsub A(a,b,s,cout);
initial
begin
      a=4'b1101;    b=4'b1000;
#10 a=4'b1001;    b=4'b1111;
#10 a=4'hF;         b=4'hA;
#20 $finish;
end
endmodule
```





8. Behavioural modelling for multibit magnitude comparator

- A comparator used to compare two binary numbers each of four bits is called a 4-bit magnitude comparator.
- It consists of eight inputs each for two four-bit numbers and three outputs to generate less than, equal to, and greater than between two binary numbers.

multibit magnitude comparator

In a 4-bit comparator, the condition of **A>B** can be possible in the following four cases.

1. If A3 = 1 and B3 = 0
2. If A3 = B3 and A2 = 1 and B2 = 0
3. If A3 = B3, A2 = B2 and A1 = 1 and B1 = 0
4. If A3 = B3, A2 = B2, A1 = B1 and A0 = 1 and B0 = 0

Similarly, the condition for **A<B** can be possible in the following four cases.

1. If A3 = 0 and B3 = 1
2. If A3 = B3 and A2 = 0 and B2 = 1
3. If A3 = B3, A2 = B2 and A1 = 0 and B1 = 1
4. If A3 = B3, A2 = B2, A1 = B1 and A0 = 0 and B0 = 1

The condition of A=B is possible only when all the individual bits of one number exactly coincide with the corresponding bits of another number.

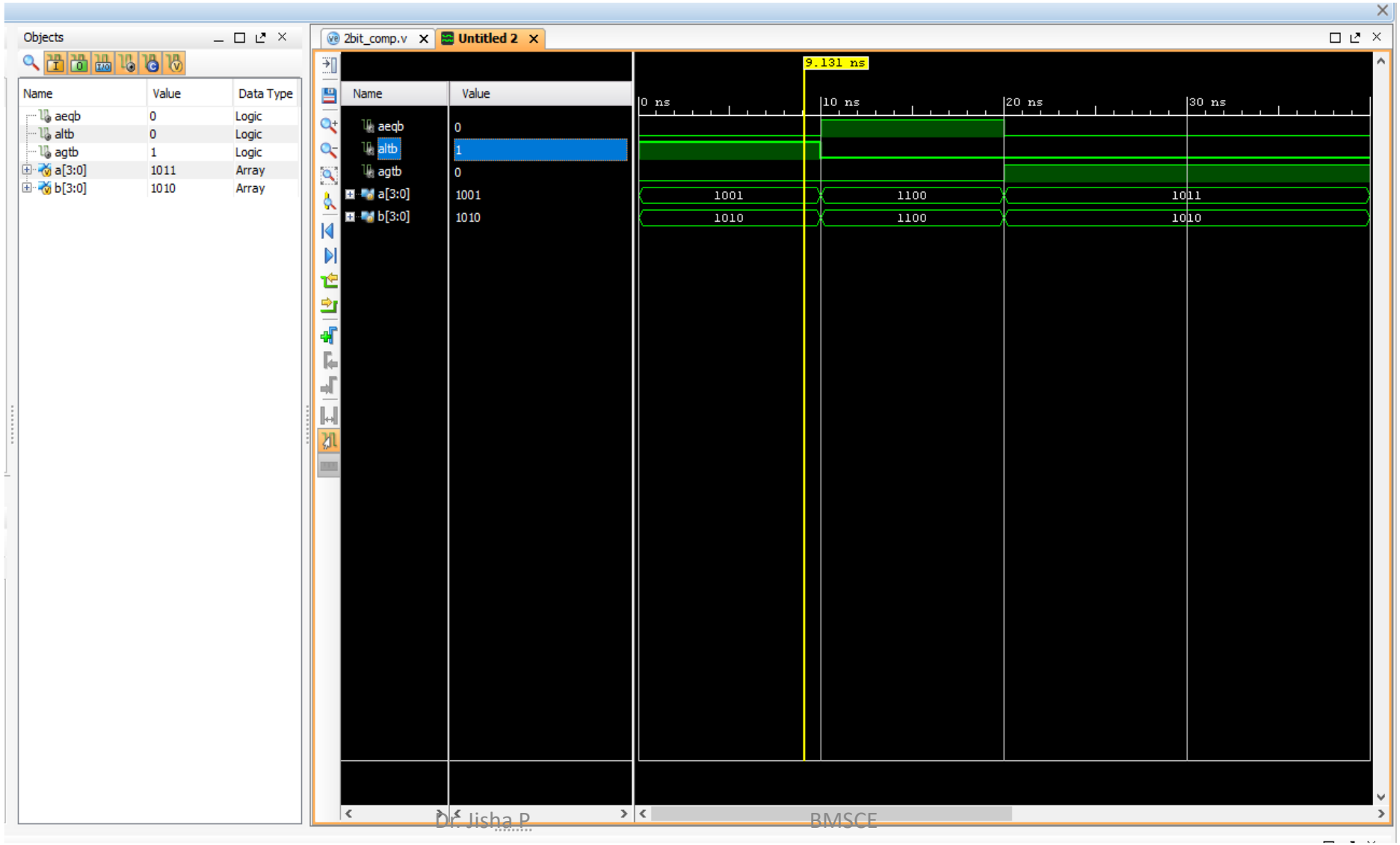
Behavioural modelling for multibit magnitude comparator

```
module comparator(a,b,aeqb,altb,agtb);
parameter N=4;
input [N-1:0]a,b;
output reg aeqb,altb,agtb;
always@(a,b)
if(a>b)
begin
altb=0;
agtb=1;
aeqb=0;
end
else if(a<b)
begin
altb=1;
agtb=0;
aeqb=0;
end
else
begin
altb=0;
agtb=0;
aeqb=1;
end
endmodule
```

Behavioural modelling for multibit magnitude comparator

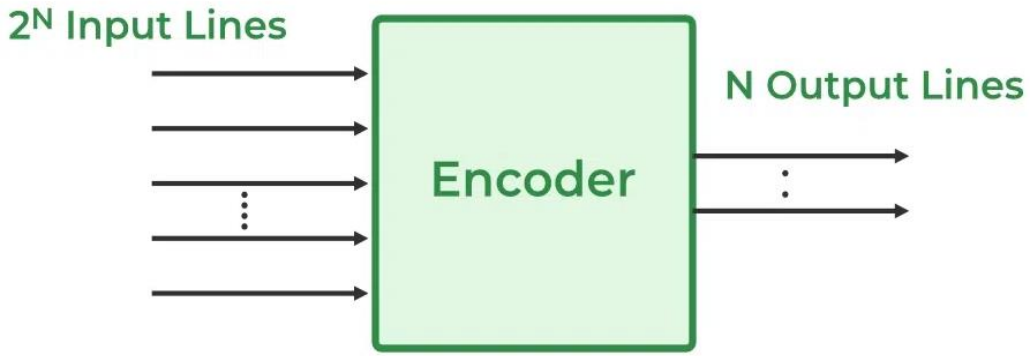
```
module tb;
wire aeqb,altb,agtb;
reg [3:0]a,b;
comparator A1(a,b,aeqb,altb,agtb);
initial
begin
a=4'd9;b=4'hA;
#10 a=4'hC;b=4'hC;
#10 a=4'hb;b=4'hA;
#20 $finish;
end
endmodule
```

Simulation output



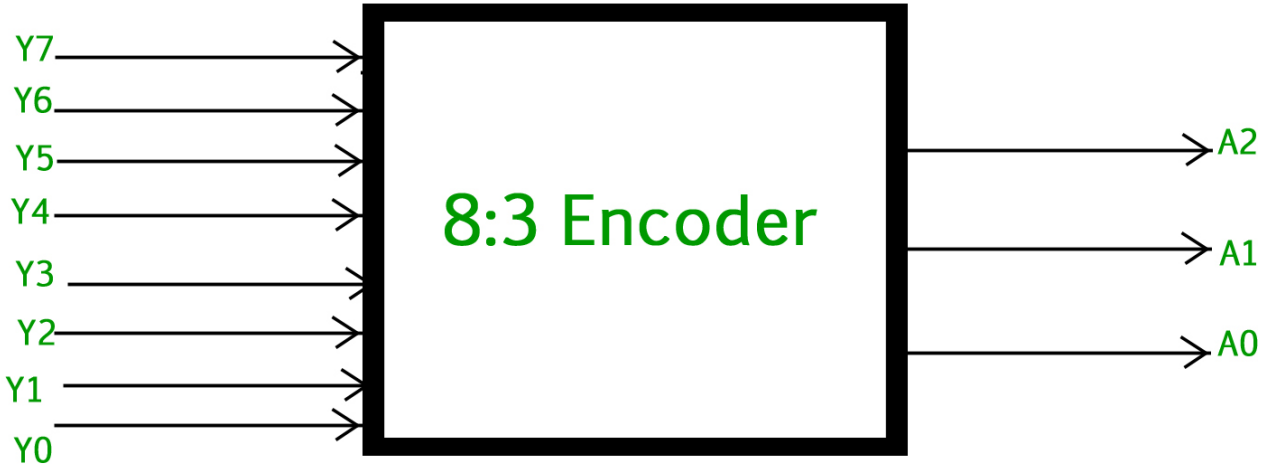
9. Behavioral modelling: Encoder without priority

- An encoder is a digital circuit that converts a set of binary inputs into a unique binary code.
- The binary code represents the position of the input and is used to identify the specific input that is active.
- Encoders are commonly used in digital systems to convert a parallel set of inputs into a serial code.
- An Encoder is a combinational circuit that performs the reverse operation of a Decoder.
- It has a maximum of 2^n input lines and ‘n’ output lines, hence it encodes the information from 2^n inputs into an n-bit code.
- It will produce a binary code equivalent to the input, which is active High.
- Therefore, the encoder encodes 2^n input lines with ‘n’ bits.



Types of Encoders:

- There are different types of Encoders which are mentioned below.
  - 4 to 2 Encoder
  - **Octal to Binary Encoder (8 to 3 Encoder)**
  - Decimal to BCD Encoder
  - **Priority Encoder**
- The 8 to 3 Encoder or octal to Binary encoder consists of **8 inputs: Y7 to Y0 and 3 outputs: A2, A1 & A0.**
- Each input line corresponds to each octal digit and three outputs generate corresponding binary code.
- The figure below shows the logic symbol of octal to the binary encoder.



The truth table for the 8 to 3 encoder is as follows.

INPUTS								OUTPUTS		
Y7	Y6	Y5	Y4	Y3	Y2	Y1	Y0	A2	A1	A0
0	0	0	0	0	0	0	1	0	0	0
0	0	0	0	0	0	1	0	0	0	1
0	0	0	0	0	1	0	0	0	1	0
0	0	0	0	1	0	0	0	0	1	1
0	0	0	1	0	0	0	0	1	0	0
0	0	1	0	0	0	0	0	1	0	1
0	1	0	0	0	0	0	0	1	1	0
1	0	0	0	0	0	0	0	1	1	1

8 to 3 Encoder- Octal to Binary Encoder.

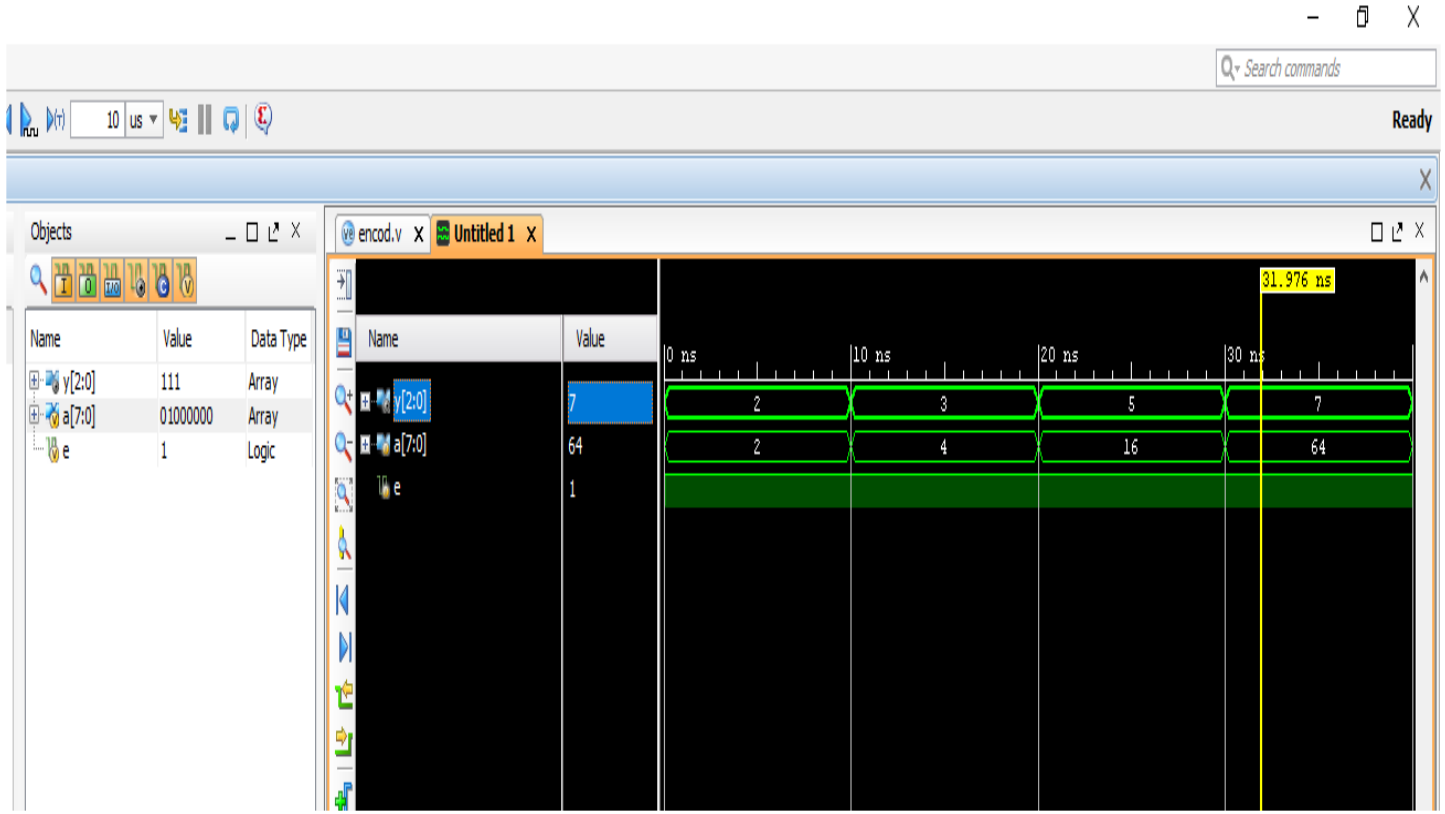
- Logical expression for A2, A1, and A0.
    - A2 = Y7 + Y6 + Y5 + Y4
    - A1 = Y7 + Y6 + Y3 + Y2
    - A0 = Y7 + Y5 + Y3 + Y1
- In 8-input lines, one input-line is set to true at a time to get the respective binary code in the output side.

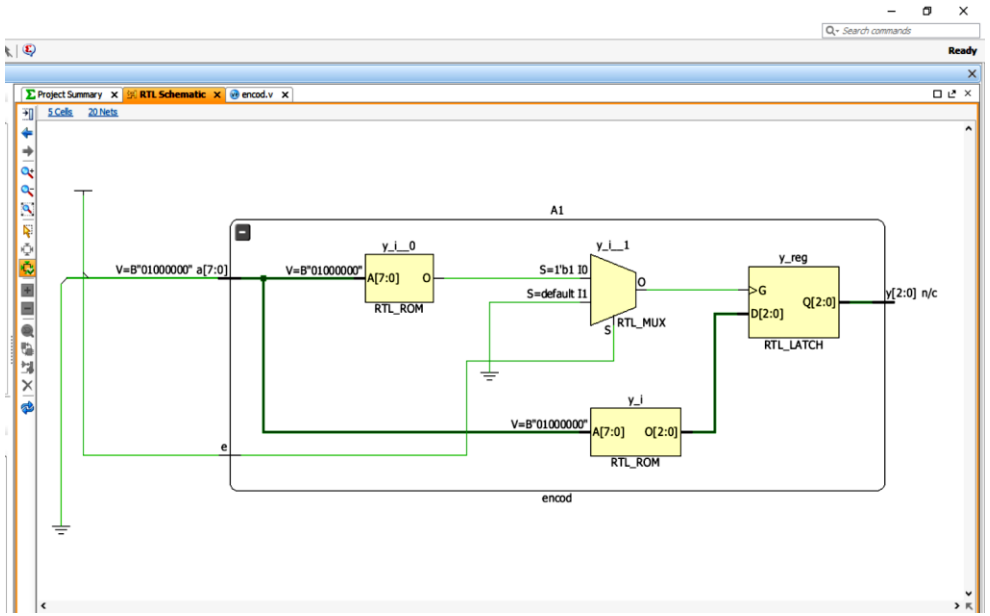
9. Behavioral modelling: Encoder without priority

```
module encod (input [7:0]a,input e,output reg [2:0]y);

always @(a,e)
begin
if(e==1)
case({a})
8'b00000001:y=3'd1;
8'b00000010:y=3'd2;
8'b00000100:y=3'd3;
8'b00001000:y=3'd4;
8'b00010000:y=3'd5;
8'b00100000:y=3'd6;
8'b01000000:y=3'd7;
8'b10000000:y=3'd8;
endcase
end
endmodule
```

```
module tb;
wire [2:0]y;
reg [7:0]a;
reg e;
encod A1(a,e,y);
initial
begin
a=8'd2;e=1;
#10 a=8'd4;e=1;
#10 a=8'd16;e=1;
#10 a=8'd64;e=1;
#10 $finish;
end
endmodule
```

Simulation & Schematic

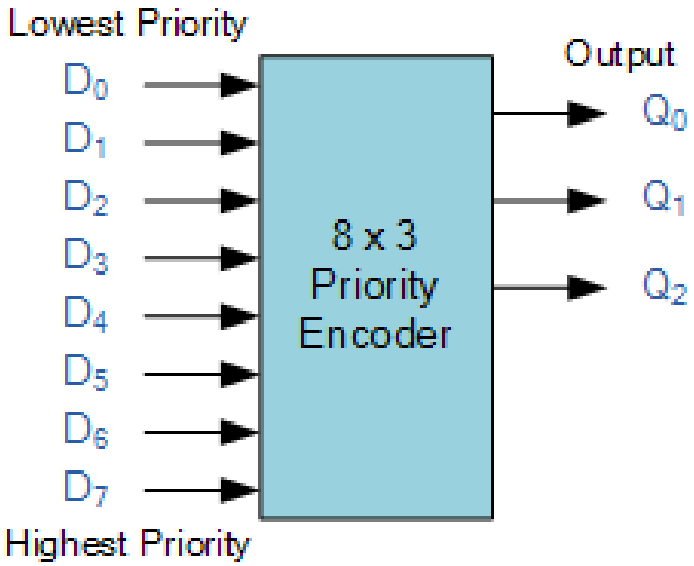




Behavioral modelling: Priority encoder

- The priority encoders output corresponds to the currently active input which has the highest priority.
- So when an input with a higher priority is present, all other inputs with a lower priority will be ignored.

- The priority encoder comes in many different forms with an example of an 8-input priority encoder along with its truth table shown below.



Inputs								Outputs		
D_7	D_6	D_5	D_4	D_3	D_2	D_1	D_0	Q_2	Q_1	Q_0
0	0	0	0	0	0	0	1	0	0	0
0	0	0	0	0	0	1	x	0	0	1
0	0	0	0	0	1	x	x	0	1	0
0	0	0	0	1	x	x	x	0	1	1
0	0	0	1	x	x	x	x	1	0	0
0	0	1	x	x	x	x	x	1	0	1
0	1	x	x	x	x	x	x	1	1	0
1	x	x	x	x	x	x	x	1	1	1

X = dont care

Behavioral modelling: Priority encoder

- The truth table for a 8-to-3 bit priority encoder is given as:

Digital Inputs								Binary Output		
D_7	D_6	D_5	D_4	D_3	D_2	D_1	D_0	Q_2	Q_1	Q_0
0	0	0	0	0	0	0	1	0	0	0
0	0	0	0	0	0	1	X	0	0	1
0	0	0	0	0	1	X	X	0	1	0
0	0	0	0	1	X	X	X	0	1	1
0	0	0	1	X	X	X	X	1	0	0
0	0	1	X	X	X	X	X	1	0	1
0	1	X	X	X	X	X	X	1	1	0
1	X	X	X	X	X	X	X	1	1	1

Where X equals “dont care”, that is it can be at a logic “0” level or at a logic “1” level.

Output Q_0

$$Q_0 = \sum(1, 3, 5, 7)$$

$$Q_0 = \sum(\bar{D}_7\bar{D}_6\bar{D}_5\bar{D}_4\bar{D}_3\bar{D}_2D_1 + \bar{D}_7\bar{D}_6\bar{D}_5\bar{D}_4D_3 + \bar{D}_7\bar{D}_6D_5 + D_7)$$

$$Q_0 = \sum(\bar{D}_6\bar{D}_4\bar{D}_2D_1 + \bar{D}_6\bar{D}_4D_3 + \bar{D}_6D_5 + D_7)$$

$$Q_0 = \sum(\bar{D}_6(\bar{D}_4\bar{D}_2D_1 + \bar{D}_4D_3 + D_5) + D_7)$$

Output Q_1

$$Q_1 = \sum(2, 3, 6, 7)$$

$$Q_1 = \sum(\bar{D}_7\bar{D}_6\bar{D}_5\bar{D}_4\bar{D}_3D_2 + \bar{D}_7\bar{D}_6\bar{D}_5\bar{D}_4D_3 + \bar{D}_7D_6 + D_7)$$

$$Q_1 = \sum(\bar{D}_5\bar{D}_4D_2 + \bar{D}_5\bar{D}_4D_3 + D_6 + D_7)$$

$$Q_1 = \sum(\bar{D}_5\bar{D}_4(D_2 + D_3) + D_6 + D_7)$$

Output Q_2

$$Q_2 = \sum(4, 5, 6, 7)$$

$$Q_2 = \sum(\bar{D}_7\bar{D}_6\bar{D}_5D_4 + \bar{D}_7\bar{D}_6D_5 + \bar{D}_7D_6 + D_7)$$

$$Q_2 = \sum(D_4 + D_5 + D_6 + D_7)$$

Behavioral modelling: Priority encoder

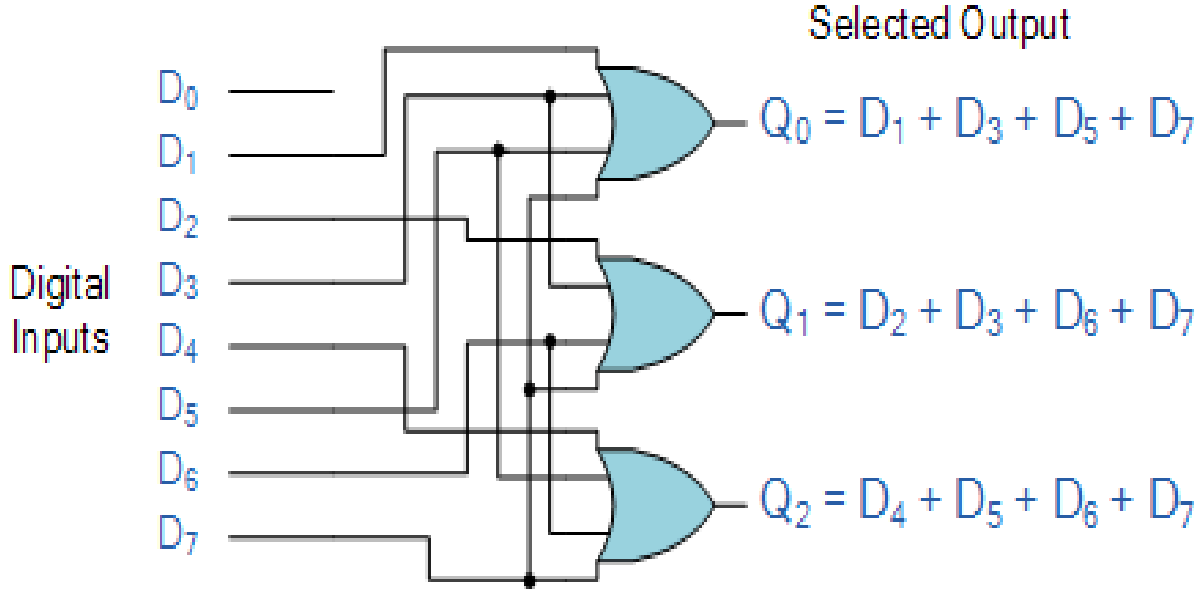
- Then the final Boolean expression for the priority encoder including the zero inputs is defined as:

- Priority Encoder Output Expression

$$Q_0 = \sum\left(\overline{D}_6\left(\overline{D}_4\overline{D}_2D_1 + \overline{D}_4D_3 + D_5\right) + D_7\right)$$

$$Q_1 = \sum\left(\overline{D}_5\overline{D}_4(D_2 + D_3) + D_6 + D_7\right)$$

$$Q_2 = \sum\left(D_4 + D_5 + D_6 + D_7\right)$$

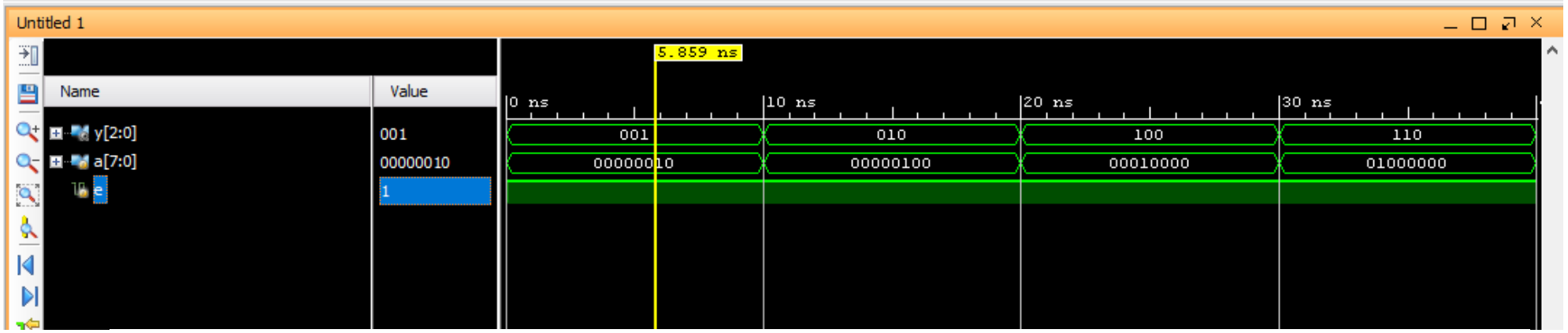


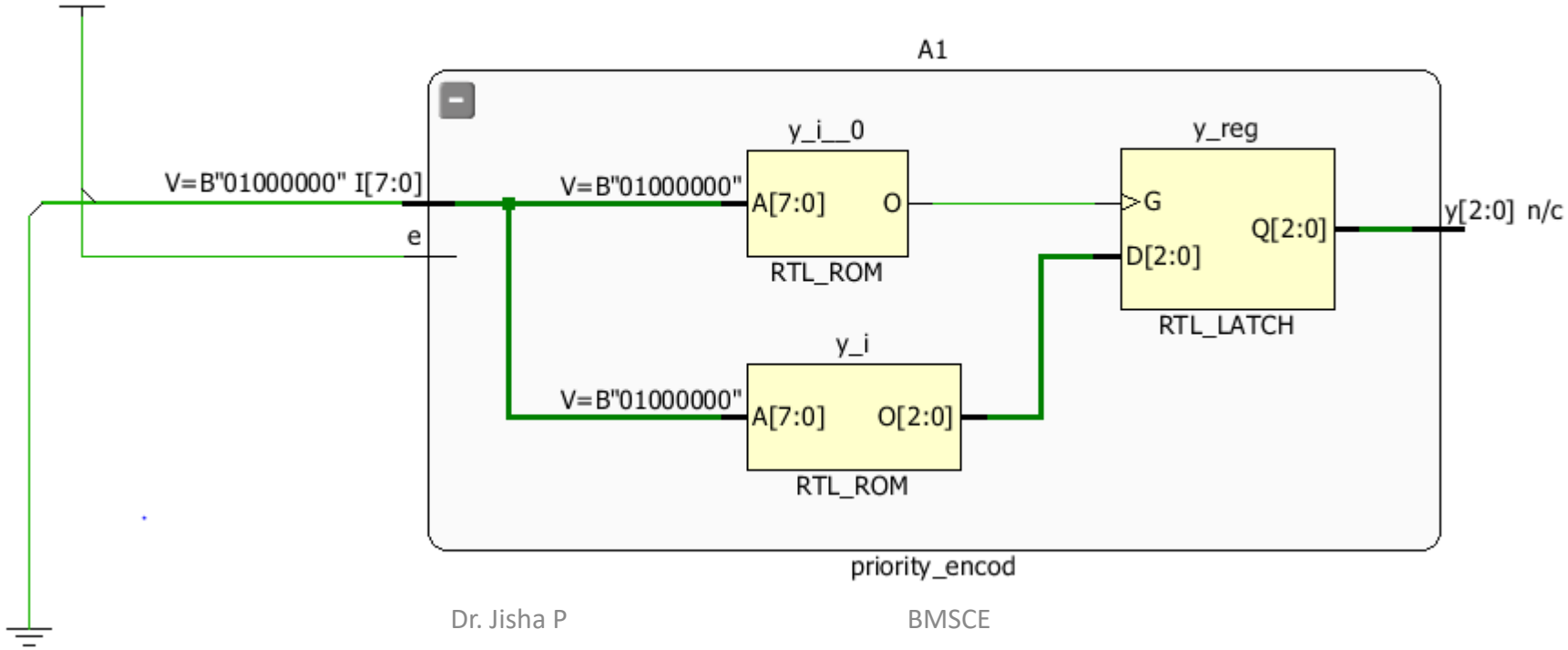
Behavioral modelling: Priority encoder

```
module priority_encod (input [7:0]I, input e,
output reg[2:0]y);
always @(I)
begin
casex(I)
8'bXXXXXXX1:y=3'b000;
8'bXXXXXX10:y=3'b001;
8'bXXXXX100:y=3'b010;
8'bXXXX1000:y=3'b011;
8'bXXX10000:y=3'b100;
8'bXX100000:y=3'b101;
8'bX1000000:y=3'b110;
8'b10000000:y=3'b111;
Endcase
end
endmodule
```

```
module tb;
wire [2:0]y;
reg [7:0]a;
reg e;
priority_encod A1(a,e,y);
initial
begin
a=8'd2;e=1;
#10 a=8'd4;e=1;
#10 a=8'd16;e=1;
#10 a=8'd64;e=1;
#10 $finish;
end
endmodule
```

Simulation & Schematic

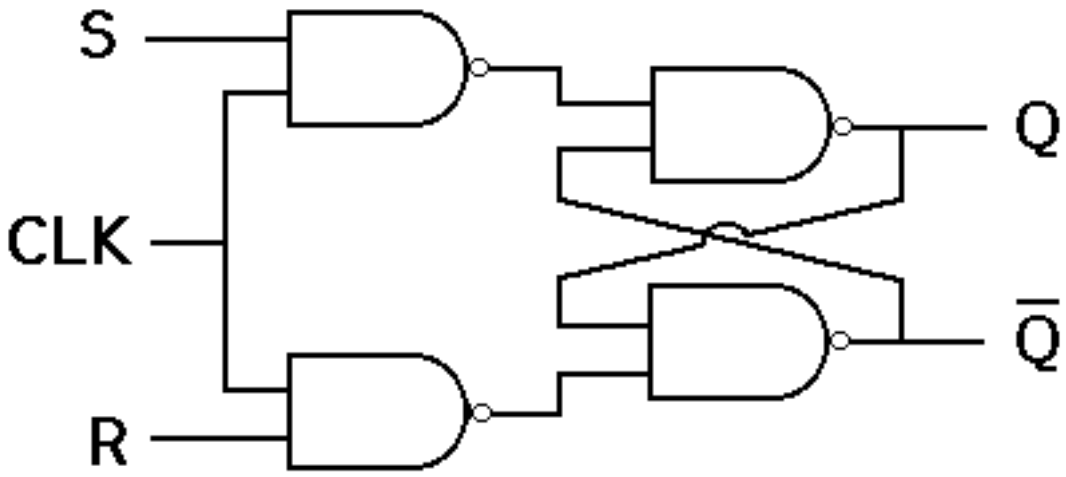




10. Behavioural modelling: SR latch, JK and D flip-flops

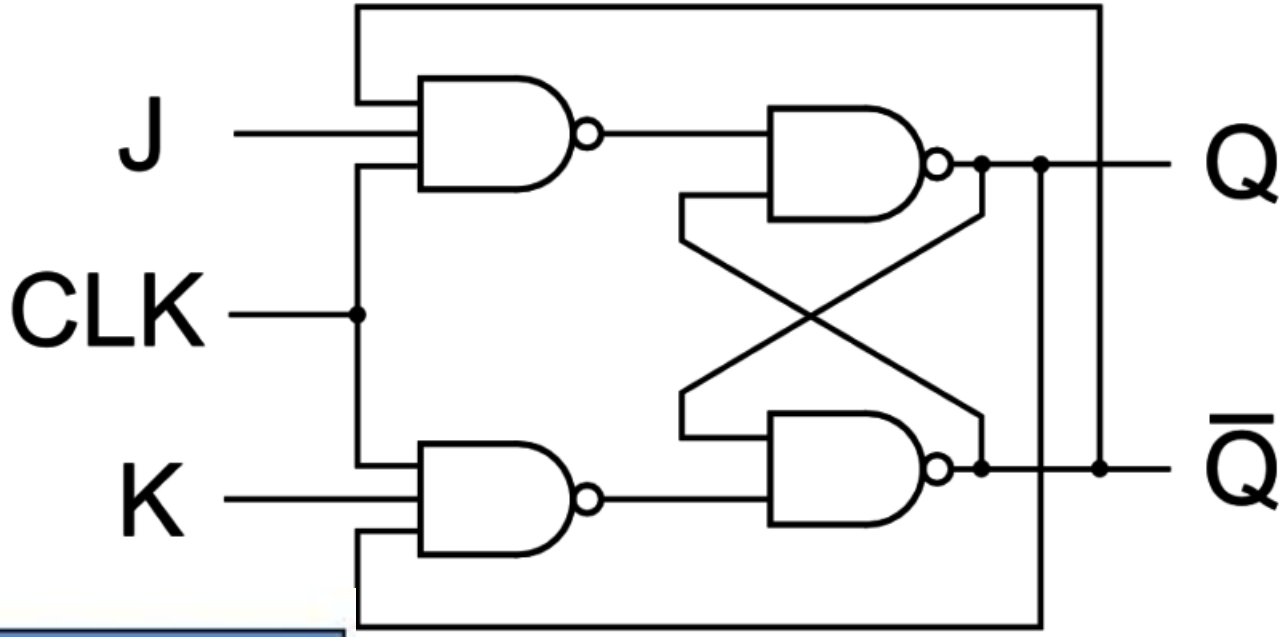
- A flip-flop in digital electronics is a circuit with two stable states that can be used to store binary data.
- The stored data can be changed by applying varying inputs.
- Flip-flops and latches are fundamental building blocks of digital electronics systems used in computers, communications, and many other types of systems. Both are used as data storage elements.
- It is the basic storage element in sequential logic.

SR Flip Flop



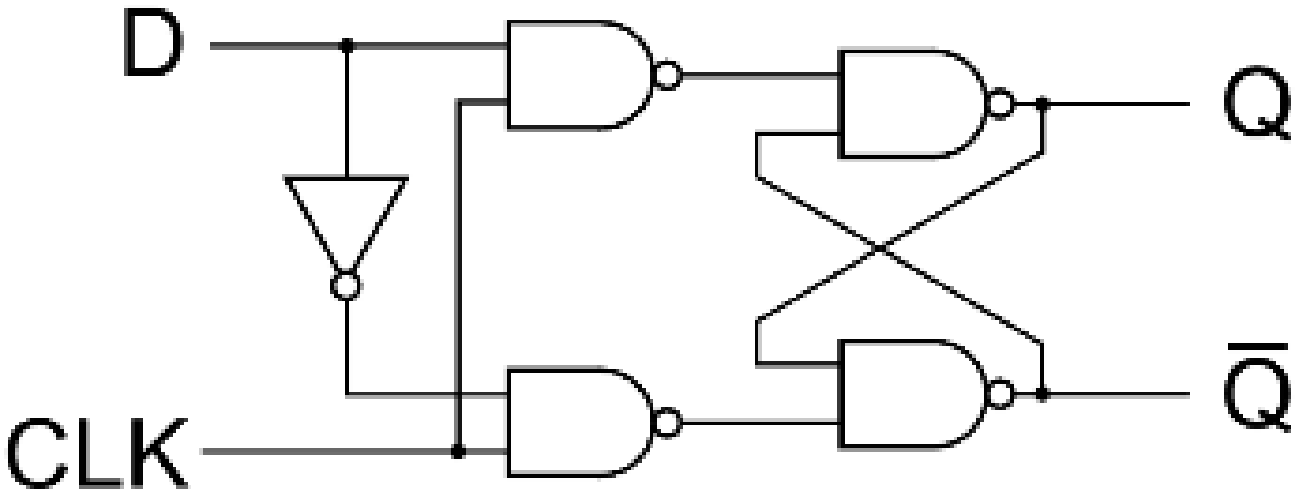
S	R	Q	Q'
0	0	0	1
0	1	0	1
1	0	1	0
1	1	∞	∞

JK Flip-Flop



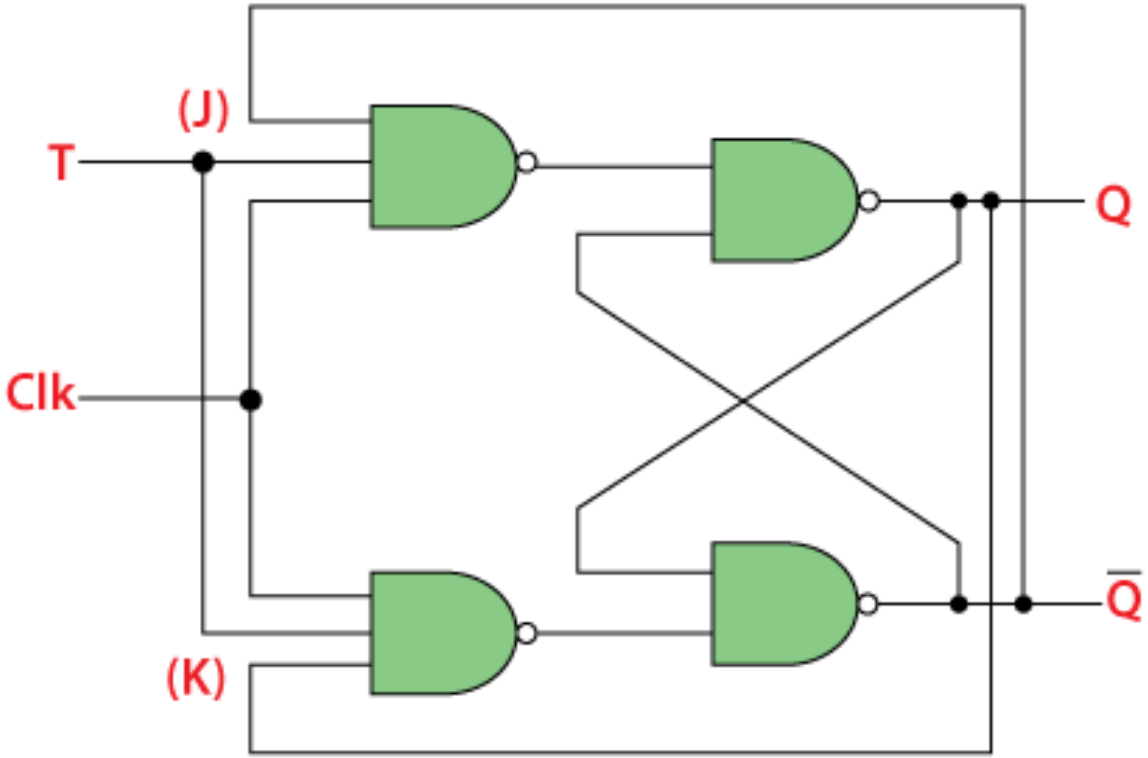
Clk	J	K	Q	Q'	State
1	0	0	Q	Q'	No change in state
1	0	1	0	1	Resets Q to 0
1	1	0	1	0	Sets Q to 1
1	1	1	-	-	Toggles

D Flip-Flop



Clock	D	Q	Q'
↓ » 0	0	0	1
↑ » 1	0	0	1
↓ » 0	1	0	1
↑ » 1	1	1	0

T Flip-Flop



Truth Table:

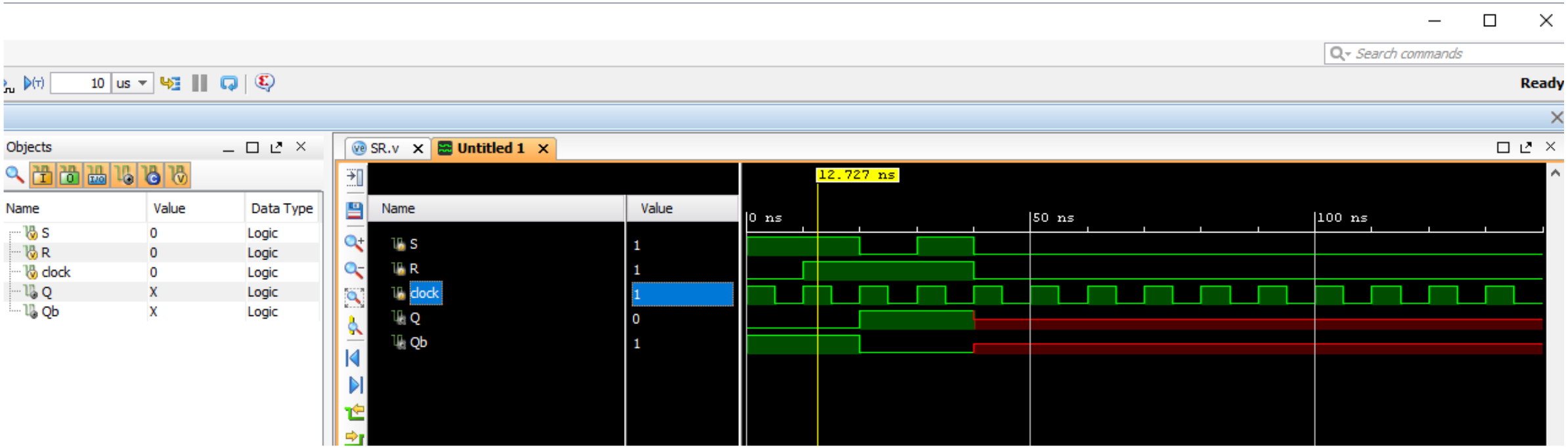
T	Q	Q (t+1)
0	0	0
1	0	1
0	1	1
1	1	0

b. SR Flip Flop

```
module SR(
input S,R,clock,
output reg Q,Qb
);
always@(posedge(clock))
begin
case({S,R})
2'b01:Q=1;
2'b10:Q=0;
2'b11:Q=Q;
2'b00:Q=1'bX;
endcase
Qb=~Q;
end
endmodule
```

```
module tb;
reg S,R,clock;
wire Q,Qb;
SR A1(S,R,clock,Q,Qb);
initial
begin
clock=1;
S=1;R=0;
#10 S=1;R=1;
#10 S=0;R=1;
#10 S=1;R=1;
#10 S=0;R=0;
#100 $finish;
end
always #5 clock=~clock;
endmodule
```

Simulation

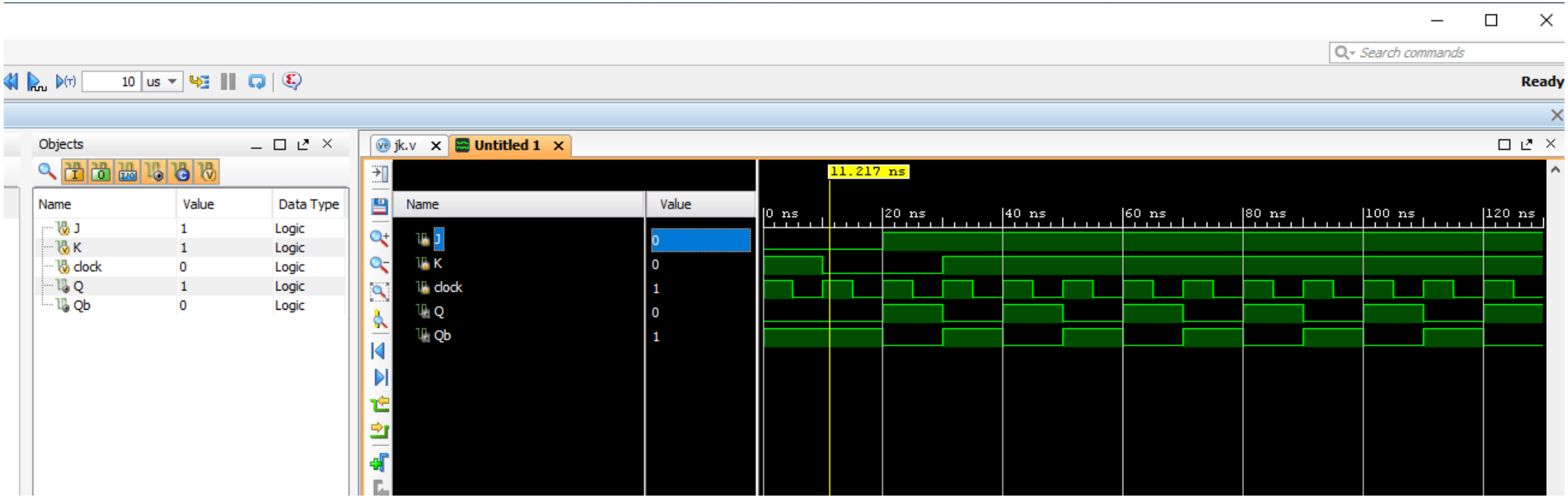

Search commands
10 us
Ready
Objects
Name Value Data Type
S 0 Logic
R 0 Logic
clock 0 Logic
Q X Logic
Qb X Logic
SR.v
Untitled 1
12.727 ns
Name Value
S 1
R 1
clock 1
Q 0
Qb 1
0 ns
50 ns
100 ns

a. JK Flip Flop

```
module JK( input J,K,input clock,
output reg Q,Qb );
always@(posedge(clock))
begin
case({J,K})
2'b01:Q=0;
2'b00:Q=Q;
2'b10:Q=1;
2'b11:Q=~Q;
endcase
Qb=~Q;
end
endmodule
```

```
module tb;
reg J,K,clock;
wire Q,Qb;
JK A1(J,K,clock,Q,Qb);
initial
begin
clock=1;
J=0;K=1;
#10 J=0;K=0;
#10 J=1;K=0;
#10 J=1;K=1;
#100 $finish;
end
always #5 clock=~clock;
endmodule
```

Simulation

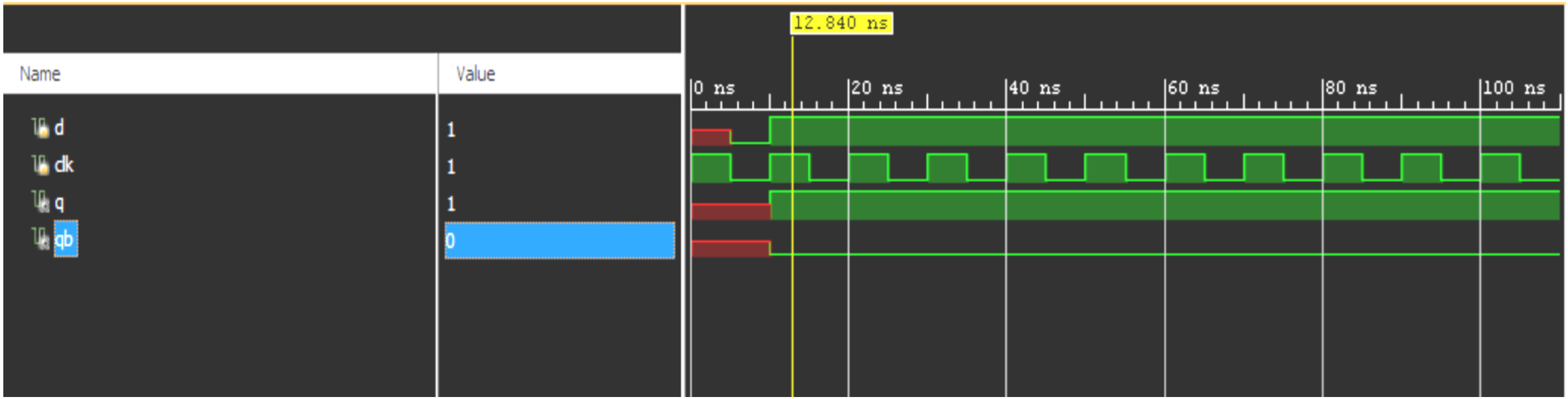

Search commands
Ready
10 us
Objects
Name Value Data Type
J 1 Logic
K 1 Logic
clock 0 Logic
Q 1 Logic
Qb 0 Logic
jk.v
Untitled 1
11.217 ns
Name Value
J 0
K 0
clock 1
Q 0
Qb 1
0 ns
20 ns
40 ns
60 ns
80 ns
100 ns
120 ns

c. D-Flip Flop

```
module dff (d,clk,q,qb);
input d,clk;
output reg q,qb;
always @ (d,clk)
begin
if(clk==1)
begin
q=d;
qb=~d;
end
end
endmodule
```

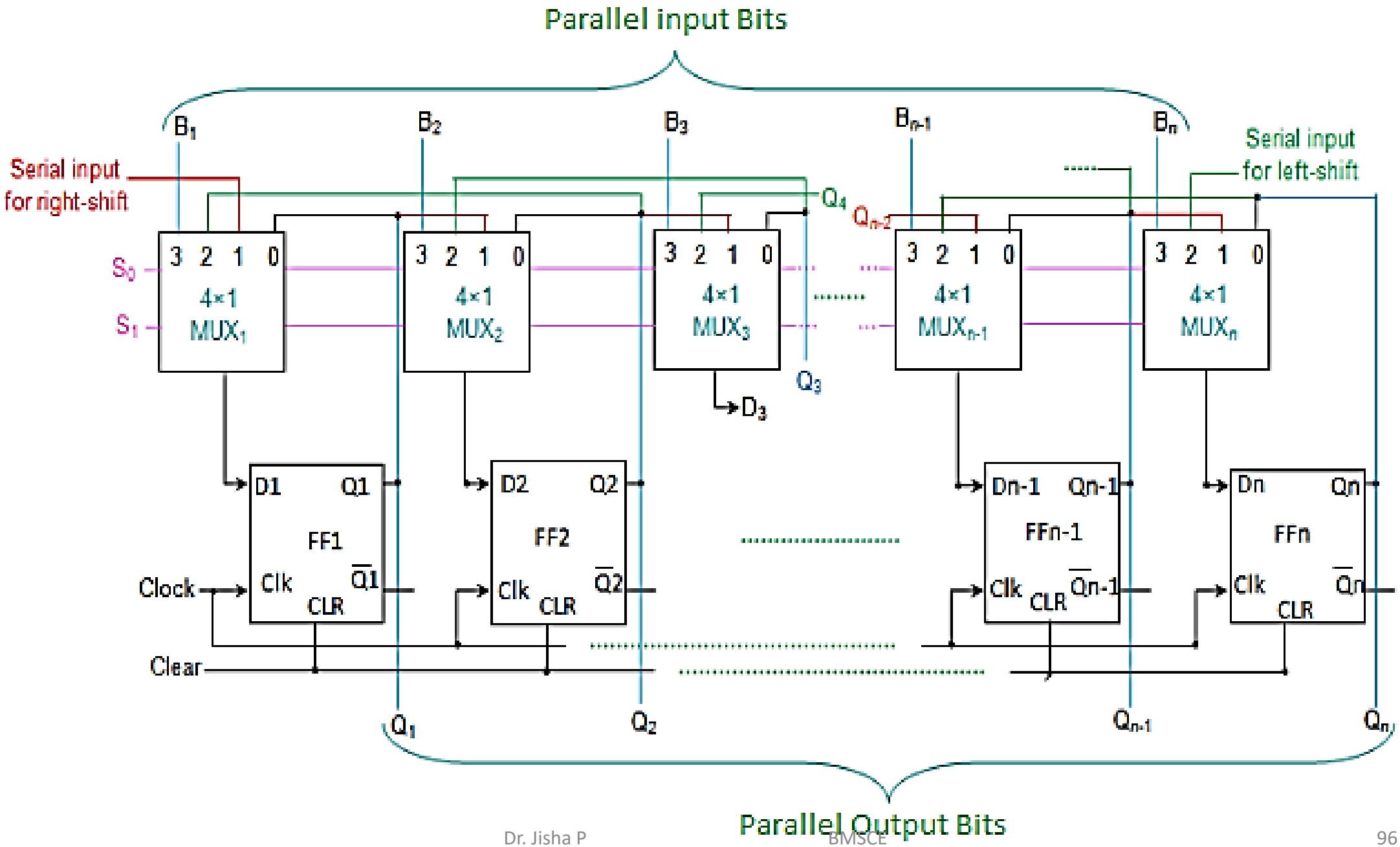
```
module dff_tb; reg d,clk; wire q,qb;

dff(d,clk,q,qb); initial begin clock=1;

#5 d=0;

#5 d=1;

#100 $finish;

end

always #5 clock=~clock;

endmodule
```

Simulation


Name
Value
d
1
dk
1
q
1
qb
0
12.840 ns
0 ns
20 ns
40 ns
60 ns
80 ns
100 ns

Universal Shift Register

- A Universal shift register is a register which has both the right shift and left shift with parallel load capabilities.
- Universal shift registers are used as memory elements in computers. A Unidirectional shift register is capable of shifting in only one direction.
- A bidirectional shift register is capable of shifting in both the directions.
- The Universal shift register is a combination design of bidirectional shift register and a unidirectional shift register with parallel load provision.

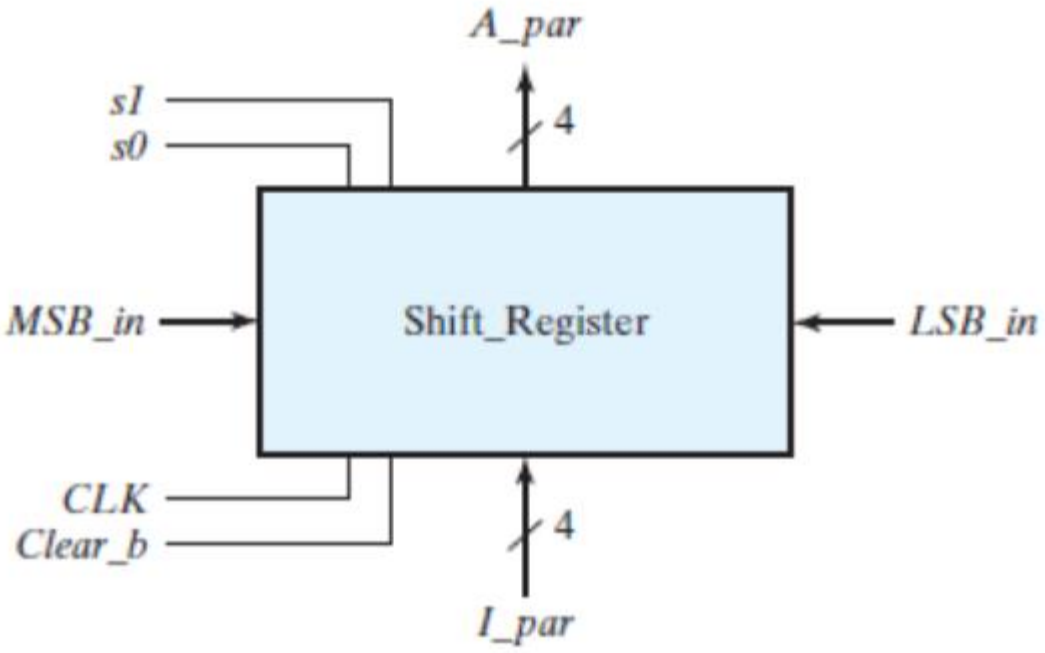

Parallel input Bits
B1
B2
B3
Bn-1
Bn
Serial input for right-shift
Serial input for left-shift
Q4
Qn-2
S0
S1
3 2 1 0
4×1 MUX1
4×1 MUX2
4×1 MUX3
4×1 MUXn-1
4×1 MUXn
Q3
D3
D1 Q1 FF1 Clk CLR Q̄1
D2 Q2 FF2 Clk CLR Q̄2
Dn-1 Qn-1 FFn-1 Clk CLR Q̄n-1
Dn Qn FFn Clk CLR Q̄n
Clock
Clear
Q1
Q2
Qn-1
Qn
Parallel Output Bits

Universal Shift Register

- Basic connections

1. The first input (zeroth pin of multiplexer) is connected to the output pin of the corresponding flip-flop.
2. The second input (first pin of multiplexer) is connected to the output of the very-previous flip flop which facilitates the right shift.
3. The third input (second pin of multiplexer) is connected to the output of the very-next flip-flop which facilitates the left shift.
4. The fourth input (third pin of multiplexer) is connected to the individual bits of the input data which facilitates parallel loading.

A n-bit universal shift register consists of n flip-flops and n 4×1 multiplexers. All the n multiplexers share the same select lines(S1 and S0)to select the mode in which the shift register operates. The select inputs select the suitable input for the flip-flops.



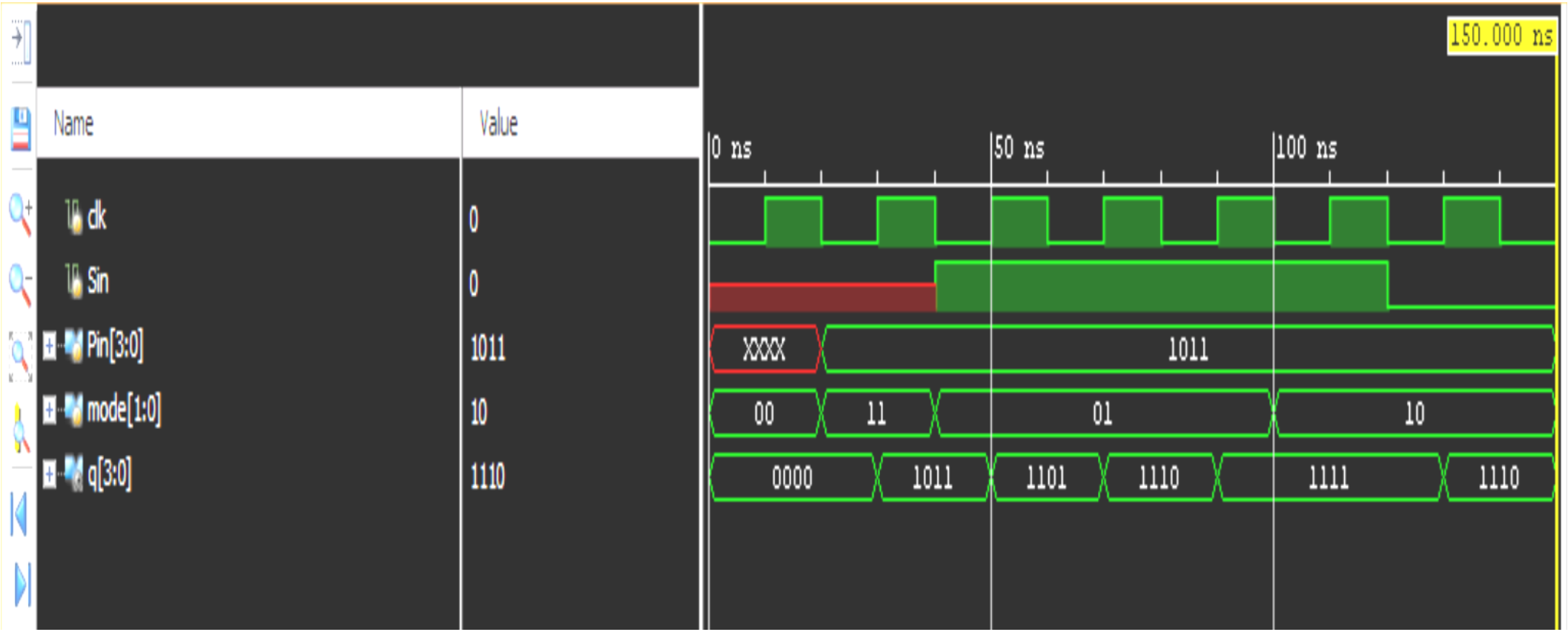
Mode Control		
s_1	s_0	Register Operation
0	0	No change
0	1	Shift right
1	0	Shift left
1	1	Parallel load

11.Behavioral Modelling: Universal Shift Register

```
module Ushift_reg( input clk, input Sin,
input [3:0] Pin, input [1:0] mode,
output reg [3:0] q );
initial q=4'b0000;
always @(posedge clk)
case(mode)
2'b00: q<=q; //hold
2'b01: q<={Sin,q[3:1]}; //right shift
2'b10: q<={q[2:0],Sin}; //left shift
2'b11: q<=Pin; //parallel load
endcase
endmodule
```

```
module Ushift_reg_tb;
reg clk, Sin;
reg [3:0] Pin;
reg [1:0] mode;
wire [3:0] q;
Ushift_reg inst1(clk,Sin,Pin,mode,q);
Initial    begin
clk = 1'b0;
mode = 2'b00;
#20 mode = 2'b11; Pin=4'b1011;
#20 mode = 2'b01; Sin=1'b1;
#60 mode = 2'b10; Sin=1'b1;
#20 mode = 2'b10; Sin=1’b0;
#30 $finish;
end
always #10 clk=~clk;
endmodule
```

Simulation


Name
Value
clk
0
Sin
0
Pin[3:0]
1011
mode[1:0]
10
q[3:0]
1110
150.000 ns
0 ns
50 ns
100 ns
XXXX
1011
00
11
01
10
0000
1011
1101
1110
1111
1110